

集中式 Config 管理系統

技術設計文件 — Web UI + Git backend

文件版本 0.23 | 2026-09-02 | 討論用途

修訂紀錄

版本	日期	主要變更
0.23	2026-09-02	修正圖 9 中「未部署」與「一致」兩條回饋線重疊且穿過方框；用語統一「登出」→「退出」（與身分輸入非認證機制的性質一致——沒有登入就沒有登出）
0.22	2026-09-02	排版 ：只有章（h2）從新頁開始，小節不另起頁；標題與其後第一個區塊綁在一起，標題不落在頁尾。 內容檢查修正三處矛盾 ：§ 3.2 仍寫「引入 Monaco 時需要打包工具」——但 § 7.5 已定案不做文字編輯器，改為「整個 v0.10.0 都不需要打包工具」；附錄 C.2 的元件評估仍以程式碼編輯器為主，重寫為表單元件評估並對齊 § 7.5.5 的結論；參數表 wireframe 的按鈕仍是「儲存並套用」，改為「儲存」（草稿）
0.21	2026-09-02	§ 7.3 導航模型改為雙向 ：原圖畫成單向，等於暗示子視圖是有去無回的頁面跳轉，與 § 7.2.1 「一頁到底」的決定矛盾。改為主畫面為核心、所有子視圖完成後回到主畫面；明訂子視圖之間沒有直接跳轉、返回時保留位置與草稿、退出為唯一出口
0.20	2026-09-02	排版 ：每章一律從新頁開始（原本 § 0、§ 2、§ 8 接在前一章之後）；標題與其後第一個區塊綁在一起，消除 5 處落在頁尾的孤行標題（page-break-after: avoid 在舊版 WebKit 不可靠）。 用語統一 ：進板→進版、退板→退版、Layer N→第 N 層；歷史畫面的篩選與列表改為行為描述，不再出現 cfg / revert 等內部代號
0.19	2026-09-02	儲存與進版分離 ：先前把「記錄與寫出原子化」誤推成「儲存＝立刻記錄並寫出」，導致一次調整三份 config 產生三次版本迭代。改為三段（編輯→儲存草稿→進版），進版為全域動作且整批原子；草稿存於編輯階段、與四種狀態正交。主畫面補上進版／退版／歷史／退出按鈕與草稿標記，身分輸入頁補上角色切換。新增 ADR-00000022，ADR-00000006 加註修訂；新增測試介面 T18
0.18	2026-09-02	圖表產生腳本納入文件包 （tools/ + figures/），可重跑、內建節點溢出與重疊檢查；記錄兩個 SVG 相容性坑（orient 屬性、height:auto）。 新增 doc/UI-ELEMENTS.md ：把 wireframe、HTML 元素、測試選取器三者綁在一起，含每個畫面的 data-testid 命名與角色差異的斷言方式
0.17	2026-09-02	§ 7 依單機前提全面修正 ：左側樹改以群組為階層（原圖誤含多機階層），群組由 config 屬性決定且可編輯儲存；搜尋框加範圍下拉；參數表補齊各型態示範，list 加順序調整； 新增「角色」概念 （一般使用者／開發者，與環境模式正交）——型別指定、白名單維護、屬性編輯限開發者；歷史改以行為描述取代內部代號。新增測試介面 T16／T17 與驗收旅程 A6
0.16	2026-09-02	術語改用中文「測試介面」 取代外來詞 seam；編號改為 T1-T15。 補上完全缺漏的驗收層 ：五個使用者旅程 A1-A5，各含失敗判準；系統層問「功能運作正確嗎」，驗收層問「使用者能達成目的嗎」——每一步都通過但步驟順序讓人卡住時，只有驗收層測得出來。覆蓋率表新增驗收欄。文件更名為 doc/TEST-PLAN.md
0.15	2026-09-02	測試介面補完 ：對照每個模組與每條流程做覆蓋率審計，補上七個原本無測試介面覆蓋的行為——清單檔寫回的原樣保證（T1 擴充）、型別推斷與 schema 骨架（T12）、編輯階段生命週期（T13，須注入時鐘）、參數索引與搜尋（T14）、啟動前置檢查（T15），以及偏離兩條處置路徑（T11 補充）。新增覆蓋率對照表與三個刻意留空的理由
0.14	2026-09-02	uid 改為匯入毫秒時間戳轉 base36 ：唯一性來自單調遞增而非機率，消除碰撞檢查與偶發失敗的測試，附帶保留納管順序。 新增 T11 瀏覽器端到端測試介面 ：介面行為改為自動化驗證，以語意屬性選取元素而非 DOM 結構；「不測前端」的範圍縮小為 DOM 結構、CSS class 與排列順序。新增 ADR-00000019
0.13	2026-09-02	§ 3.7 新增測試項目定義 ：實作採測試先行，而先前只定義了測試「放哪」與「驗收什麼」，沒有定義「測什麼」。新增十個測試介面及其驗證行為、明確不測的項目、三個反模式與迴圈規則；完整清單另附 doc/TEST-PLAN.md（狀態為提案，需逐項確認）。新增 ADR-00000018
0.12	2026-09-02	§ 0.5 新增 ADR 結構 lint ：實測對照組 27 份 ADR 結構一致性 100%，但其 lint 只檢查編號不檢查結構——一致性靠自律維持。本專案於 v0.1.0 即納入結構檢查（缺 Serves／必要段落／Status 非法 fail，缺 Alternatives warn），並補入 v0.1.0 驗收檢查點。 § 0.2 補充不變式 4 與 5 的分工說明
0.11	2026-09-02	§ 7 全章重寫 ：參數欄位表取代文字編輯器（原設計等同線上 YAML 編輯器，與 vim 無異）；左側樹＋雙擊展開＋多 config 並列；參數層級搜尋；開發／部署模式；單一編輯階段鎖；白名單介面維護。 § 8 版本重排 ：每版皆含 backend + frontend，目標版本改為 v0.10.0，v1.0.0 由維護者自行標記。 § 0.4 early return 改為通則而非超標補救。 § 2.3 commit 改用 Conventional Commits，範圍只放 uid。 § 3.2 補 framework 與前端的分層說明、FastAPI 選型理由、容器組成（不採 sidecar）。 § 3.3 src 扁平化、web 移入 src。 § 3.3.5 新增模組解耦的介面契約。 § 3.4 澄清 schema 驗證所有格式。術語全面統一
0.10	2026-09-02	對齊基準改以模板 main 分支的初始腳本為權威，release 與既有 downstream 降為參考；補釘住 commit 的要求
0.9	2026-09-02	§ 3.3 依實際 downstream 重寫（三種擁有權、啟動腳本契約、hooks 與命令組）；子目錄原則改為依數量決定
0.8	2026-09-02	正文聚焦目標版本，其餘移入附錄 A／B／C；§ 3.5 介面設計定案（CLI 作為 API client、REST + 動作子資源）
0.7	2026-09-02	修正 SVG 箭頭方向（orient 屬性不相容導致所有向左向上箭頭錯誤）；圖表編號連號化；狀態名稱中文化
0.6	2026-09-02	§ 4.3 身分設計（uid／name／hostname 三欄位）；§ 8.1 版本能力矩陣；§ 0.1 詞彙表
0.5	2026-09-02	第 0 章拆為原則專章，對齊策略與測試架構移至第 3 章
0.4	2026-09-02	新增第 0 章原則（不變式、設計原則、ADR 規範、衝突優先序）
0.3	2026-09-02	全部圖表改為向量繪製；新增 UI wireframe；每張流程圖補文字敘述
0.2	2026-09-02	清單檔格式改為 TOML；新增驗證機制與正規形式白名單
0.1	2026-09-02	初版：問題陳述、架構決策、核心流程

目錄

正文——v0.10.0 範圍

0. 原則（詞彙 | 不變式 | 設計原則 | 實作規範 | ADR | 衝突優先序）
1. 問題陳述與目標
2. 架構決策紀錄
3. 系統架構（服務拓撲 | 技術選型 | 目錄結構 | 介面設計 | 測試架構 | 測試項目定義）
4. 資料格式與 config 清單檔規格
5. 核心流程（納管 | 進版 | 退版 | 差異偵測 | 狀態機）
6. 驗證機制與值的正規形式
7. UI／UX 設計
8. 版本里程碑與驗收檢查點（v0.1.0 – v0.10.0）

附錄——v0.10.0 之外

- A. 導入組織共用容器模板
- B. v0.10.0 之後的功能規劃
- C. 參考專案評估

0. 原則

本章的地位與讀法

本章是後續所有設計決策的判斷依據。當文件其餘部分與本章衝突時，以本章為準。

本章分兩層：**不變式**（§ 0.2）是產品必須恆常維持的性質，任何 ADR 都不得違反；**設計原則**（§ 0.3）是不變式未涵蓋、本專案特有的判斷依據。其下為實作規範、決策記錄規範，以及原則彼此衝突時的判斷順序。

每條規則**完整寫出而非引用外部文件**——讀本文件的人不需要先讀過其他資料，也應能完整理解與執行。

0.1 詞彙

本章使用以下名詞。完整定義見後續章節，此處先給最小可用的說明，避免先用後定義。

名詞	意義	詳見
來源 repo	一個 git repo，存放所有被管理的 config 內容。系統認定它是唯一正確的版本	§ 3.4
目標位置	config 實際被程式讀取的路徑，例如 <code>/opt/robot/config/nav2.yaml</code> 。內容由系統寫出	§ 2.1
config 清單檔	來源 repo 內的一份檔案，記錄「管了哪些 config、各自的目標位置與權限」	§ 4.3
納管	把一份已存在於磁碟上的 config 納入管理。過程不改變檔案內容	§ 5.1
寫出	把來源 repo 的內容寫到目標位置，並設定權限。英文常稱 apply	§ 5.2
進版／退版	套用新設定／回復到先前版本	§ 5.2、§ 5.3
偏離	目標位置的內容與來源 repo 不一致，代表有人繞過介面直接改了檔案	§ 5.4
原樣解析	一種讀寫方式：改動其中一個值，其餘的註解、引號樣式、順序原封不動。英文稱 round-trip	附錄 C.3

0.2 產品不變式

以下是本專案必須恆常維持的性質。**任何 ADR 都不得違反**；違反的設計不是取捨，是缺陷。

不變式 1 | 每個容器只跑一個服務

整個系統由數個容器組成（frontend、backend，未來可能加入反向代理與背景工作），**每個容器只負責一個服務**。服務之間透過明確的網路介面溝通，不共用程序空間。

推論一：未來若實作定期背景掃描，**不得在 backend 容器內另起排程程序**——那會使該容器變成兩個服務。應走主機排程呼叫 CLI，或另起一個容器。

推論二：開發用與測試用的容器是「一次性工作階段」而非服務，其生命就是那次互動或那次測試。**自動重啟策略只發給真正的服務**；對工作階段容器設自動重啟會導致結束後無限重啟，離不開。

不變式 2 | 絕不靜默失敗

任何錯誤、缺失或不相容的設定都必須**大聲且提早**失敗，絕不是仍然顯示成功的靜默跳過。契約在做實事之前先自我驗證。

這是本專案最核心的一條，幾乎所有設計決策都是它的推論：

設計	如何服務此不變式
重複 key 一律報錯	多數 parser 靜默取最後一筆，造成「明明寫了卻沒生效」。錯誤訊息必須列出 key 名稱與 所有 出現行號
歧義值報錯而非猜測	<code>no</code> ／ <code>08</code> ／ <code>1.10</code> 的解析結果依 parser 而異；猜錯即是靜默改壞配置
否決 symlink 部署方案	symlink 被 rename 寫入覆蓋後靜默失效，檔案仍在、內容看似正常，但已脫離管理
偏離偵測	使「繞過流程」這件事本身發出訊號，而非無聲累積
原子寫入 + commit／apply 成對	避免半殘檔案，也避免「已改未記錄」的無主狀態
config 清單檔五項完整性檢查	兩筆 <code>target</code> 指向同一路徑時，結果取決於 寫出順序——典型的靜默 bug
驗證未通過即拒絕 commit	後端是 gate；不做「警告後仍然放行」
錯誤訊息指名檔案與行號	「格式錯誤」等同於沒有訊息，使用者無從行動

此不變式的通用檢驗

任何「系統自動幫使用者處理掉」的設計都需重新檢視：**它是在消除麻煩，還是在消除訊號？**

不變式 3 | 可擴充為多實例（前瞻約束）

資料模型不得寫死無法跨實例共存的字面值。這是一條**前瞻保證**：它的存在是為了讓未來擴充到多台機器時，不會撞牆而必須回頭改既有結構。

本專案的對應：`config-list.toml` 目前假設單機。未來加入機器維度時，必須能**加入而不需改動既有結構**。因此現在即確保：

- `groups` 是多對多 tag 而非樹狀階層——機器可作為一個 tag 維度直接加入。
- 身分拆成 `uid`（不變）／`name`（可讀）／`hostname`（實例）三層——機器維度已由 `hostname` 承擔，跨機比對時以 `name` 分組即可。

這兩點在 § 4.3 已如此設計；此處記錄其真正理由不是「彈性」，而是這條前瞻保證。

不變式 4 | 預設值落向安全

當一個預設值有「安全 vs 方便」的張力時，預設落向**安全**，較危險或較緊的選項是 opt-in。

決策	安全的一邊（預設）	opt-in 的一邊
偏離的處置	不自動處置，等待人為判斷	手動選擇覆蓋或納入
驗證失敗	拒絕 commit	第 3 層 可 override，需填理由
提權寫入	服務不以 root 執行	sudoers 白名單限定特定操作
解除納管	保留 target 檔案	刪除需二次確認
退版	revert，歷史不改寫	reset 不提供給 UI
容器網路	host	bridge 為 opt-in

不變式 5 | 預設值由兩個問題決定，不由個人偏好

一個功能開關要不要預設打開，問兩個問題：

- 1. 打開它，會不會讓原本正常運作的東西變得不正常？會 → 預設關。
- 2. 忘記打開它，會不會出事？會 → 預設開。

只有兩者同時成立（不會弄壞任何東西，且忘了開會出事）才預設開；其餘一律預設關。

為何要固定成規則：開關累積到十幾個之後，若每次都憑當下判斷，預設值會逐漸失去一致性。這條規則讓「該不該預設開」變成可以被 review 檢查的問題。

與「預設值落向安全」的分工

不變式 4 說的是**傾向**（衝突時往哪邊倒），這一條說的是**程序**（怎麼判斷）。前者在有明確安全風險時適用，後者處理更常見的情況——沒有明顯安全風險，只是「開著比較方便」。兩條並存不重複：先問這兩題得到答案，若兩題的結果衝突或都不明確，再用不變式 4 決定往哪邊倒。

開關	預設	依規則的判斷
開啟畫面時掃描差異	開	只讀不寫，不會弄壞任何東西（✓）；忘了掃描，使用者可能對著已被別人改過的資料操作（✓）
定期背景掃描 + 通知	關	會產生通知流量、需要額外程序（✗，第一題就不過）
語法與結構驗證	開	正確的內容不會被擋（✓）；缺了它就是這個系統存在的理由消失（✓）
跨欄位規則檢查	開，但只警告	檢查本身無害（✓），但可能誤判——所以是「開且警告」而不是「開且阻擋」
記住此裝置的身分	關	多人共用同一台機器時會誤用他人身分（✗）

不變式 6 | 一個真實來源，向外傳播，而非平行維護 N 份副本

這條在本專案有**雙重身分**。

作為工程約束：任何一份資料只有一個 owner。同一份設定不得同時由兩套機制產生或維護。

作為產品命題：config-manager 這個產品，就是把這個模式從「程式碼傳播」套用到「config 傳播」。

核心不變式（本專案的產品命題）

Repo 是唯一真實來源，目標位置的檔案是 render 產物。任何「以磁碟現況為準」的邏輯都違反此原則。

不變式 7 | 維持嚴謹且與業界對齊的測試門檻

承諾本身是不變式；具體的分類法與覆蓋率工具是可置換的機制。本專案的測試框架見附錄 A.1。

不變式 8 | 身分與命名一次解析，來自檔案

命名的分歧來自**明確的、檔案記錄的** override，絕不來自環境變數，也絕不來自目錄名的意外。

為何固定：這兩種失效模式都是無聲的，而且代價是已經看起來完成的工作。從目錄名推導的專案名，會讓兩份 checkout 靜默共用容器與網路，只靠「剛好取了不同名字」這個意外維持隔離；從環境變數取得的名字，對讀 repo 的人不可見，也活不過一個新開的終端機。

本專案的對應：config-list.toml 是每份 config 身分的唯一來源。

- uid 是系統產生的穩定識別碼，**納管後永不變更**——路徑、名稱、hostname 皆可改，身分不變。
- target、format、permissions 一律來自 config 清單檔，**不從檔案系統現況或環境變數推斷**。副檔名不可作為 format 的來源（.yaml / .param 等變體不可靠）。
- 權限資訊必須明文記錄——git 只保存 executable bit，靠 git 推斷等同於靠意外。

不變式 9 | 文件由程式碼推導，不在旁邊複製一份

可從樹推導出來的數字或清單，**在被閱讀時計算**，不存進一份需要有人手動維持一致的追蹤檔案。文件裡放的是任何 generator 都產不出來的東西：意圖、理由、以及一個東西為何是這個形狀。

為何固定：一份看起來權威但內容已過期的文件，就是不變式 2 的靜默失敗被寫進文件裡——而且比報告中的錯誤數字更糟，因為讀者無從分辨哪些句子是推導出來且已過期的、哪些是撰寫的且仍然有效。

對本文件自身的影響見 § 0.7。

出處

以上九條與組織共用容器模板所定義的不變式相容，日後導入該模板時可直接對照（見附錄 A.1）。另有一條「開發與現場以相反方式供裝」在模板導入後才適用，列於附錄 A.2。

0.3 設計原則

以下是上述不變式未涵蓋、本專案特有的原則。每條標註其服務的不變式。

N-1 | Raw 檔案的讀者是程式，不是人

服務：不變式 2。

人透過介面閱讀解析後的內容。**raw 檔案的實際讀者是 parser、腳本與 agent**——因此格式選擇的標準是「程式能不能可靠解析」，不是「人看起來順不順」。

這條讓「嚴謹」有了可操作的定義

嚴謹 = 機器可靠解析 = 同一份檔案，任何符合規格的 parser 都得到相同結果。這不是兩個要互相權衡的目標，是同一件事。TOML 之所以優於 YAML，正是因為它沒有需要 parser 猜測的地方（§ 4.2）。

具體體現：config 清單檔採 TOML；正規形式白名單拒絕所有「解析結果依 parser 版本而異」的寫法（§ 6.3）。

邊界

「讀者是程式」不等於「人永遠不會看」。git diff、code review、現場緊急處理讀的都是 raw。因此：**格式選擇以機器可解析為準，但不刻意產生人難讀的 raw。**原樣解析保留註解與格式，正是這條邊界的體現。

N-2 | 偵測與提示，不做自動修正

服務：不變式 2。

系統可以指出問題並建議修正方式，但**不替使用者決定意圖**。此原則獨立列出，因為它是實作時最容易被「順手優化」掉的一條。

具體體現：no 不自動改為 "no" ——若原意確為布林值，自動修正即是改壞配置且無人知曉；尾隨空白標示但不自動移除；偏離 提供兩種相反處置由人判斷，而非單一「修復」按鈕。

推論到實作：config 清單檔或規則檔中無法辨識的欄位是格式錯誤，須**指名檔案與行號並失敗**——絕不靜默跳過，也絕不靜默降級為較寬鬆的行為。

N-3 | 保證來自後端，介面只是入口

服務：不變式 2。

UI 本身不提供任何保證。所有驗證與稽核邏輯在後端；前端的即時回饋僅是體驗優化。

推論：不得以「使用者只能透過 UI 操作」作為安全假設——因此 偏離偵測是必要機制而非附加功能。

N-4 | 核心邏輯不依賴 I/O

服務：不變式 7。

狀態判定、驗證、config 清單檔完整性等邏輯必須可在無檔案系統的情況下測試。依賴方向為 api → core → io 單向（§ 3.3）。

此原則同時對應「深模組」設計：一個小介面覆蓋其私有實作細節。**不得為了達成 1:1 的測試對應而拆分原始碼檔案**——那會把實作攤在檔案接縫上，讓模組變淺，是本末倒置（附錄 A.1）。

N-5 | 每個介面操作都有 CLI 對等

服務：不變式 1（背景工作走主機排程呼叫 CLI）、不變式 7（可測試性）。

不存在只能透過瀏覽器完成的功能。理由：可自動化、可寫整合測試、UI 故障時仍有操作路徑、除錯時可精確重現、未來的定期掃描不需在容器內加程序。

N-6 | 可逆性優先

服務：不變式 4。

不做不可逆的預設行為。破壞性操作需明確的二次確認，且應保留回退路徑。

具體體現：退版使用 revert 而非 reset，歷史不被改寫；解除納管不刪除 target；納管不修改原始檔案內容。

0.4 實作規範

性質

本節是不變式 7 底下的**可置換機制**，不是不變式。違反時於 code review 討論，不需 ADR。**所有規範必須可由工具檢查**——無法自動檢查的規範等同不存在。

規範	閾值	檢查工具	理由
巢狀深度	≤ 3 層	<code>pylint max-nested-blocks=3</code>	計入 <code>if / for / while / try / with</code> ，非僅迴圈。與 SonarQube 預設一致。 此為上限而非目標 ——攤平是通則，見下方
循環複雜度	≤ 10	<code>ruff C901</code>	McCabe 指標，業界通用閾值。巢狀合格但分支過多的函式仍難測試
函式長度	≤ 50 行	<code>ruff PLR0915</code>	超過通常表示職責不單一
參數個數	≤ 5	<code>ruff PLR0913</code>	超過時改傳 <code>dataclass / pydantic model</code>
型別註解	<code>core/</code> 全覆蓋	<code>mypy --strict</code>	對應 N-4： <code>core/</code> 是地基，型別錯誤成本最高
禁止吞錯誤	無裸 <code>except</code> ：	<code>ruff E722、BLE001</code>	直接對應不變式 2
覆蓋率	<code>core/</code> ≥ 85%	<code>pytest-cov</code>	僅對 <code>core/</code> 要求行覆蓋率； <code>io/ / api/</code> 以 <code>integration / system</code> 層覆蓋

例外處理的具體要求

- 不得捕捉後僅 `pass` 或僅 `log.debug`。捕捉即代表有處理策略，否則應向上拋出。
- 面向使用者的錯誤訊息必須包含三要素：發生什麼、在哪裡（檔案／行號／欄位）、該怎麼改。「格式錯誤」不合格。
- 不得將驗證失敗轉為警告後繼續，除非該層級的設計即為警告（第 3 層）。

執行方式
納入 <code>just test</code> 與 CI gate， 未通過不得合併 。閾值可調整，但調整需修改設定檔並在 PR 中說明理由——使放寬本身成為可見的決策，而非個案 <code># noqa</code> 。

0.5 決策記錄規範（ADR）

ADR 檔名與編號事後改動需要重命名整個目錄並修正所有交叉引用，因此格式在第一份 ADR 寫下之前就要定案。

項目	規範
檔名	<code>doc/adr/NNNNNNNN-<slug>.md</code> ——8 位數補零編號 + kebab-case slug。 <code>README.md</code> 是唯一豁免的非 ADR 檔
註冊表	檔案系統就是註冊表。不維護另一份號碼清單
編號 lint	掛在 <code>just test</code> ：重號 fail 、檔名不符 <code>^[0-9]{8}-.+\.md\$</code> fail 、跳號僅 warn （跳號可能是刻意的）。訊息須指名是哪個檔案／號碼
結構 lint	同一支檢查再驗結構（見下方方塊）：缺 <code>> 服務：</code> 、缺 <code>Context / Decision / Consequences、Status</code> 不在允許值內 → fail ；缺 <code>Alternatives</code> → warn
結構	<code># 句子式標題 → > 服務：... → - **Date:** / - **Status:** / - **Relates to:** → ## Context / ## Decision / ## Alternatives / ## Consequences</code>
<code>> 服務：</code>	必填 。回指所服務的不變式編號、產品目標、或明寫「機制，無對應不變式」。這使每個決策必須掛在產品目標上，而非掛在「當時覺得這樣比較好」
Status	<code>Accepted / Rejected / Superseded by ADR-NNNNNNNN</code>
修訂方式	可在原檔內修訂：以 <code>**Amendment (#issue, YYYY-MM-DD):**</code> ... 段落記在原檔。只有推翻才開新 ADR 並將舊者標 <code>superseded</code>
撰寫時機	決策做成當下，不事後補。事後補寫的 ADR 幾乎必然遺漏當時真正的驅動因素與被否決的選項

結構 lint 必須存在，不能只靠自律
抽查組織共用容器模板的 ADR 目錄作為對照：27 份全部有 <code>> Serves：</code> 回指、全部有 <code>Context / Decision / Consequences</code> 、25 份有 <code>Alternatives</code> ——結構一致性極高。但檢視其編號 lint 的實作（128 行）發現它只檢查檔名格式、重號與跳號，完全不碰內容結構；掃過全部 20 個檢查驅動，沒有任何一支驗證 ADR 的段落。
也就是說：那個 27/27 是靠人的自律維持的，沒有任何機制擋下第 28 份漏掉 <code>Serves</code> 的 ADR。
這正是不變式 2 要防的形態——CI 顯示綠燈，但契約已經破了，而且要到很久以後有人去讀那份 ADR、發現它沒說在服務什麼，才會知道。 本專案的結構 lint 在 v0.1.0 就要寫 ，成本是在既有的編號檢查裡多幾行 <code>grep</code> 。

為何 Alternatives 是 warn 而非 fail
對照組有 2/27 沒有此段落。硬擋會讓既有內容無法通過，而且確實存在「只有一種做法」的決策。但缺少它是一個訊號——沒有比較過的決策，通常只是偏好——所以要出現在輸出裡，只是不阻擋合併。這與既有 lint 對「跳號」的處理一致：印出來，不擋。

為何是「原檔內修訂」而非「一律不可變」
常見的建議是「ADR 一經接受即不可修改」。實務上這會失真：當一個決策本身沒變、但它當初論證所用的案例已經不存在時，若強制開新 ADR，讀者必須自行拼湊兩份文件才知道舊的哪一段還算數。在原檔加註「別照上面的 Context 段落去實作，去讀 ADR-NNNNNNNN」更準確。 區分準則：決策沒變 → 原檔加註；決策被推翻 → 新 ADR + superseded。

已建立的 ADR

以下 22 份已隨本文件一併產出（見隨附文件包 `doc/adr/`），不需重建。新決策從 `00000023` 續接。

編號	標題	服務
0001	以 ADR 記錄決策，並強制 Serves 回指	機制
0002	來源 repo 是唯一真實來源，目標是產物	不變式 6
0003	以複製 + 寫出取代符號連結部署	不變式 2
0004	清單檔採用 TOML	不變式 2（經 N-1）
0005	退版產生新紀錄，不改寫歷史	不變式 2 + N-6
0006	記錄與寫出是單一原子操作	不變式 2
0007	變更紀錄採 Conventional Commits，範圍放 uid	機制
0008	偏離只呈現不自動處置	不變式 2 + N-2
0009	CLI 是介面同一組端點的 client	N-5
0010	資源走 REST，動作走子資源	機制
0011	核心層不做 I/O，模組可獨立呼叫	不變式 7（經 N-4）
0012	身分拆為 uid/name/hostname	不變式 8
0013	介面以型別欄位編輯參數，不編輯檔案文字	不變式 2
0014	單一入口頁，一次一個編輯階段	不變式 2
0015	兩個容器，不加 sidecar	不變式 1
0016	對齊模板 main 分支，釘住 commit	不變式 6 + 8
0017	ADR lint 檢查結構，不只編號	不變式 2
0018	測試只寫在事先議定的介面上	不變式 7
0019	瀏覽器端到端測行為，以語意屬性選取	不變式 7
0020	角色與環境是正交概念	不變式 4 + 5
0021	人工型別指定存入 schema，逐欄位可清除	不變式 2
0022	儲存草稿與進版分離（ADR-0006 加註）	不變式 2

0.6 原則衝突時的優先序

原則之間必然衝突。以下優先序用於實際判斷，**上位勝出**。



圖 1 | 原則優先序與其依據

層級	依據	典型衝突與判斷
P1	不變式 2、8	「原子化 commit + apply 讓操作變慢」→ 仍維持原子化。資料正確性與可追溯性不因效能或便利讓步
P2	不變式 2、N-2	「重複 key 報錯會讓既有檔案匯不進來」→ 不放寬報錯，改以納管階段警告、修改階段硬擋的過渡設計（§ 6.3）。 解法是設計出兩者兼顧的路徑，而非降低保證
P3	不變式 4、5	「開啟時自動掃描會拖慢載入」→ 仍預設 ON，因它符合兩分支規則（透明且缺席是 footgun）；效能問題以非同步載入解決
P4	不變式 2（經 N-1）	「TOML 的巢狀語法沒有 YAML 直覺」→ 選 TOML。嚴謹度優先於書寫習慣
P5	—	「驗證未通過時停用儲存按鈕會讓使用者困惑」→ 改善錯誤訊息與按鈕旁的說明，而非放行儲存

使用方式

此優先序用於**否決不合理的權衡**，不是用來合理化「難用」。若某個決定犧牲了 P5，應同時記錄「為了滿足哪一條上位原則」——若答不出來，那就不是原則衝突，只是設計沒做好。

0.7 本文件自身的地位

不變式 9 要求「可從程式碼推導的內容不進入追蹤文件」。這不是整潔癖好：一份看起來權威但已過期的文件，就是不變式 2 的靜默失敗被寫進文件裡，而讀者無從分辨哪些句子仍然有效。

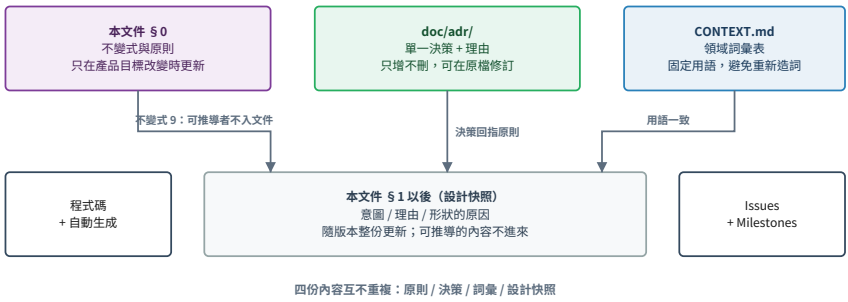


圖 2 | 文件分工

文件分工

文件	放什麼	更新時機
本文文件 § 0	不變式與原則	只在產品目標改變時。機制改變不動它
doc/adr/	單一決策 + 理由 + 被否決的選項	只增；可在原檔修訂；推翻才開新號
CONTEXT.md	領域詞彙表，固定用語	新增術語時。避免重新造詞
doc/TEST-PLAN.md	測試介面與驗收旅程	新增測試介面需更新並確認
doc/UI-ELEMENTS.md	wireframe ↔ HTML ↔ 測試選取器對照	改介面時三者一起改
tools/ + figures/	圖表產生腳本與輸出	改圖就是改腳本再重跑，不手改 SVG
本文文件 § 1 以後	設計快照：意圖、理由、形狀的原因	隨版本整份更新

建議現在就建立 **CONTEXT.md**：`config-list.toml` / `target` / `source` / 納管 / 進版 / 退版 / 偏離 / 寫出 這些詞在本文件中已有固定用法，若不寫下來，第二個維護者會重新造詞。成本極低，且屬「現在做零成本」那一類。

本文中屬於推導內容、應在程式碼存在後移除的部分

以下章節為**討論階段的暫定內容**。一旦對應的程式碼或設定檔存在，這些段落即應刪除並改為指向真實來源——保留它們就是在維護一份必然會與樹不一致的副本。

章節	內容	應替換為
§ 0.6	lint 閾值表	<code>pyproject.toml</code> / <code>.pylintrc</code> 。文件只留「為何設這些閾值」
§ 0.5	測試目錄樹	實際的 <code>test/</code> 目錄。文件只留三軸的定義與鏡射規則
§ 3.3	專案目錄樹	讀取時以 <code>tree</code> 產生，或直接看 repo
§ 3.5	REST API 端點表	FastAPI 自動產生的 OpenAPI 文件
§ 4.3	<code>config</code> 清單欄欄位表	<code>pydantic model</code> 的 <code>schema</code> ，可自動渲染
§ 6.3	正規形式白名單表	<code>config-repo/.lintrc.toml</code> (§ 6.4)。文件只留「為何拒絕這些形式」
§ 8	版本里程碑與驗收檢查點	GitHub Milestones + Issues。驗收條件屬 issue 的 acceptance criteria

為何現在仍保留

在程式碼尚不存在時，這些內容是討論的必要材料——沒有它們無法對規格取得共識。但它們的**存續期限是「到對應程式碼落地為止」**，不是永久。建議在實作 PR 中一併刪除，並於 PR 描述引用本節。

不屬於推導內容、應永久保留的是：§ 0 全章、§ 1 問題陳述、§ 2 決策紀錄（在 ADR 建立後改為索引）、§ 5 流程圖與敘述、§ 6.1–6.2 驗證策略的理由、§ 7 UI 設計意圖、附錄 B 未來規劃、附錄 C 選型評估。

1. 問題陳述與目標

1.1 現況痛點

- **分散**：config 散落系統各處（`/opt`、`/etc`、ROS package 內），無集中清單。
- **無版控**：修改後無法判斷改了什麼、誰改的、何時改的，也無法回到先前狀態。
- **修改易出錯**：YAML／JSON 的型別與縮排陷阱多，直接以文字編輯器修改風險高（見 § 6.3）。

1.2 目標

- 提供單一 Web 介面，作為 config 修改的唯一正規入口。
- 以 Git 為版控核心，所有修改留下 commit 紀錄（含修改者 email）。
- 支援進版與退版。
- 修改前經自動驗證，攔截型別與格式錯誤。
- 能偵測繞過介面的直接修改。

1.3 v0.10.0 排除範圍

- 多機器分發／fleet 管理。
- 檔案權限強制鎖定（唯讀）— 以流程規範取代技術強制。
- 使用者認證與權限分級 — email 僅作為 commit author。
- ROS 2 install space 自動偵測 — 由使用者自行指定（見 附錄 B.2）。
- JSON Schema 驗證與 Form view（見 附錄 B.1）。

2. 架構決策紀錄

2.1 檔案部署機制

方案	機制	評估結果
A. Symlink 否決	檔案搬入 repo，原位置建立 symlink 指回 repo。	否決。 (1) Docker container 內解析不到 repo 路徑會斷鏈，須額外 mount 且路徑須一致，此約束會傳染至每個新 service。(2) write-temp-then-rename 類寫入會將 symlink 替換為一般檔案並靜默失效。(3) repo 與目標為同一份檔案，無法區分修改來源。
B. Bare repo 原地追蹤 否決	<code>git --git-dir=... --work-tree=/</code> ，檔案不搬動。	否決。 零侵入是優點，但 work-tree 指向根目錄使 <code>reset</code> 與 branch 切換語意危險，僅能逐檔 checkout，不利整批進退版。
C. Copy + Apply 採用	Repo 為唯一真實來源，透過 寫出至目標位置成為一般檔案。	採用。 Docker 完全無感；不受 rename 寫入影響；被覆蓋後重新寫出即修復；權限可於寫入時設定；repo 為正常 working tree，git 操作全為原生語意。

核心不變式

Repo 是唯一真實來源，目標位置的檔案是 render 產物。

2.2 其他決策

議題	決策	理由
config 清單檔格式	TOML	使用者透過 HTML 介面閱讀解析後的內容，不直接讀 raw，故註解的權重下降；此時唯一標準是嚴謹度。嚴謹度排序為 TOML > JSON > YAML（詳見 § 4）。
是否禁止 vim 直接編輯	不做技術強制	權限鎖定會擋到緊急除錯與自動回寫流程。改以流程規範 + 差異偵測；繞過流程屬作業疏失，由紀錄追溯。
退版實作	<code>git revert</code>	<code>reset</code> 使中間 commit 從歷史消失，損失稽核價值。歷史噪音問題以 commit 前綴規範解決（§ 2.3）。
config 清單檔形式	單一檔案	單機、使用者少，單檔可一眼掌握全貌且易做分群過濾。多人並行造成 conflict 時再折。
分群模型	多對多 tag	config 歸屬本質交叉（同時屬某台車與某子系統）。tag 支援多視角，樹狀僅能擇一。
Schema／規則位置	置於 config repo 內	Schema 隨程式演進；同版控可確保退版時一併回退，避免版本錯配產生假警報。
解除納管語意	停止管理，目標檔案保留	解除納管不應改變系統當前行為。歷史由 git 保留，不需軟刪除欄位。
Commit 與寫出	原子化	不保留「已改未套用」中間狀態，使「目標 ≠ 來源」語意唯一確定為「繞過介面修改」。

2.3 Commit 訊息規範

採用 **Conventional Commits** 形式 `<類型>(<範圍>): <說明>` ——與組織既有慣例一致。類型描述做了什麼操作，範圍是對哪一份 config。

範圍欄位放什麼

範圍只放 uid，不放可讀名稱。理由是 uid 永不變、name 與 hostname 都可改（§ 4.3）——若把可讀名稱寫進 commit 訊息，改名之後歷史就查不全：舊 commit 用舊名、新 commit 用新名，同一份 config 的歷史被切成兩段。

可讀性由 diff 本身提供。commit 的檔案路徑已經是 `files/amr01/nav2_params.yaml`，看 `git log --stat` 一眼就知道改的是哪一份；訊息裡再寫一次是重複，而且是會過期的那一份。

格式

類型	範例	用途
<code>import</code>	<code>import(mfz3k9q1): navigation-params@amr01</code>	納管新檔案。此時 uid 剛產生，說明欄記錄當時的可讀名稱作為起點
<code>cfg</code>	<code>cfg(mfz3k9q1): 調整 max_vel_x 至 0.8</code>	透過介面修改內容
<code>revert</code>	<code>revert(mfz3k9q1): rollback to a3f19c2</code>	退版
<code>adopt</code>	<code>adopt(mfz3k9q1): 納入現場調整</code>	把偏離的目標內容納回來源
<code>meta</code>	<code>meta(mfz3k9q1): 移至 perception 群組</code>	只改清單檔欄位，內容未動
<code>unmanage</code>	<code>unmanage(mfz3k9q1): 移交 ROS 套件管理</code>	解除納管

為何要有類型前綴

讓歷史可以被過濾。退版以 `revert` 實作（§ 2.2），歷史會累積退版紀錄；純看 `git log` 分不出「這份 config 真正被改過幾次」與「被退過幾次」。有了類型，介面的歷史檢視可以預設只顯示 `cfg` 與 `adopt` ——即實際改變內容的變更——其餘可選擇性展開（§ 7.6）。

此規範自 **v0.1.0** 起強制。前綴的成本接近零，但事後補做要重寫全部歷史。

3. 系統架構與實作結構

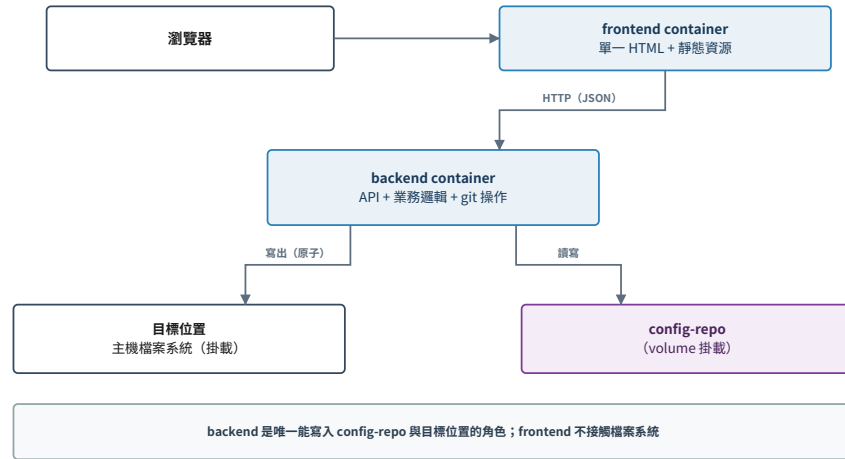


圖 3 | 服務拓撲與資料流向

3.1 Repo 分離

本系統涉及兩個獨立 git repo，職責與生命週期不同：

Repo	內容	說明
config-manager (程式碼)	Web UI、Backend、部署、測試	本系統原始碼，結構見 § 3.3。
config-repo (資料)	被管理的 config、config 清單檔、schema	純資料 repo，由本系統讀寫。其 commit 歷史即 config 的變更稽核紀錄。結構見 § 3.4。

為何分離

兩者變更頻率、權限需求、備份策略、稽核對象皆不同。混用會使 config 的變更歷史被程式碼 commit 稀釋，退版時也可能誤動程式碼。

3.2 技術選型

層	選擇	理由
Backend 語言	Python 3.11+	round-trip parser 生態最完整 (ruamel.yaml、tomlkit、configobj 皆為 Python)；與既有 ROS 2 工具鏈一致，未來接 rclpy 做 param introspection 無語言邊界。
Web framework	FastAPI	型別註解即 API schema，自動產生 OpenAPI 文件；非同步支援足以應付檔案 I/O；pydantic 可直接用於 config 清單檔的結構驗證。
Git 操作	subprocess 包裝 git CLI	不使用 GitPython/pygit2。理由：本系統的 git 操作種類少且固定，CLI 行為與人工操作完全一致，除錯時可直接複製指令重現；函式庫的抽象在 revert、log 過濾等場景反而增加不確定性。
前端	單一 HTML + 原生 JS	v0.10.0 的畫面是清單表格、按鈕與差異顯示，以 fetch 呼叫 § 3.5 的端點即可，不需框架也不需打包流程。此決定不影響 backend——後端一律回 JSON，前端形態改變不牽動它。§ 7.5 已定案以型別欄位表編輯參數（不做文字編輯器），表單元件採用可直接引入、無建置流程的函式庫（附錄 C.2），因此整個 v0.10.0 都不需要打包工具。
容器組成	frontend + backend 兩個容器	每個容器一個服務（不變式 1）。backend 掛載 config-repo 與需管理的目標路徑，是唯一能寫入檔案系統的角色；frontend 只供應靜態資源。不使用 DinD。
服務定義	.setup.conf	導入模板前為手寫 compose.yaml；導入後改由 .setup.conf 描述並由模板渲染（附錄 A）。自建版本保持最小，不長出主機偵測等功能

3.2.1 「Web framework」與「前端 HTML + JS」不是同一層

兩者常被混為一談，但它們處理的是完全不同的事：

層	跑在哪	負責什麼
前端 HTML + JS	使用者的瀏覽器	畫出畫面、接收點擊與輸入、把使用者的意圖變成一個網路請求送出去。它碰不到檔案系統，也碰不到 git——瀏覽器的安全模型不允許
Web framework	backend 容器內	接收那個網路請求、對應到一支函式、執行實際工作（讀寫檔案、跑 git、驗證內容）、把結果包成回應送回去

換句話說：前端決定「畫面長什麼樣」，framework 決定「請求進來之後怎麼被處理」。兩者透過 § 3.5 的端點連接，各自可以獨立更換——前端從單一 HTML 換成 SPA，backend 不需要改；backend 從 FastAPI 換成別的，前端也不需要改，只要端點契約不變。

3.2.2 為何是 FastAPI

Python 的 web framework 主要三個候選：

候選	特點	對本專案的評估
FastAPI	型別註解即介面定義；自動產生 OpenAPI 文件；內建請求／回應驗證	採用 。本專案已用 <code>pydantic</code> 定義 <code>config</code> 清單檔的資料模型（§ 4.3），FastAPI 直接吃同一份模型做請求驗證， 不需要維護第二份定義 。而 CLI 是 API client（§ 3.5.1），自動產生的 OpenAPI 文件即是 CLI 與介面共同的契約
Flask	最小、最自由；生態成熟	驗證與文件都要自己接。本專案的錯誤回應必須結構化（§ 3.5.3），自己接等於重造 FastAPI 已有的東西
Django	全功能；內建 ORM、後台、認證	過重。本專案沒有資料庫（狀態全在 git 與檔案系統），Django 的核心價值用不上

這個選擇與 UI／UX 無關，可以先定

UI 尚未定案不影響此決策：framework 處理的是「請求進來怎麼辦」，與畫面長相無關。反過來說也成立——**先定 framework 不會限制 UI 的可能性**，因為 backend 一律回 JSON（§ 3.5.1），前端要怎麼呈現完全自由。

唯一會被 UI 影響的是端點的粒度假（例如參數層級的即時驗證需要一支 `/validate`），而那是端點設計問題，不是 framework 選擇問題。

3.2.3 容器組成：要不要加 monitor 與 log 容器

常見的「一個模組 = app + monitor + log 三個容器」是 **sidecar 模式**，源自 Kubernetes：Pod 是部署單位，同一個 Pod 內的容器共用網路與磁碟命名空間，因此把日誌收集器與指標匯出器放在應用旁邊很自然。

本專案不加，理由有三

- **Docker 的日誌模型不需要 log 容器**。容器把日誌寫到 `stdout`／`stderr`，由 Docker 的 logging driver 收集——這是 Docker 官方的標準做法。額外開一個 log 容器是在重造 driver 已經做的事。
- **monitor 應該是主機層共用，不是每個 app 一份**。單機上跑一份指標收集器涵蓋所有容器，比每個應用配一個省得多。每個 app 一組會讓容器數量隨應用數線性增長，而它們做的是同一件事。
- **compose 沒有 Pod 這層**。sidecar 在 Kubernetes 有意義，是因為 Pod 把它們綁成一個排程與生命週期單位。純 compose 沒有這個抽象，三個容器就是三個獨立的容器，運維面積是三倍而收益接近零。

何時才該加：應用無法把日誌寫到 `stdout`（例如第三方案式只會寫檔案）、或需要在應用旁邊做協定轉換。本專案兩者皆非。

因此容器組成維持 **frontend + backend 兩個**。日誌落地由模板的執行期腳本處理（§ 3.3.3），存活監看同理，兩者都不需要額外容器。

須確認的前提

技術選型假設本系統為**單一主機、單一使用者同時操作**。若未來需多人同時編輯，需另行設計鎖定機制（目前無此需求）。

3.3 專案目錄結構

佈局參考組織共用容器模板，不是照抄。模板規定的是介面契約（哪些路徑歸模板、哪些歸專案、任務入口的形狀）；其餘由本專案依實際需要決定。

3.3.0 對齊基準

權威來源：模板 `main` 分支的初始化腳本

對齊依據是模板 `main` 分支 `dist/script/base/init.sh` 中的安裝路徑清單函式——它以明文列出初始化時裝入 downstream 的每一個路徑，是目前對 downstream 佈局最完整、最不需要推論的定義。

其餘皆為**參考**，不是依據：最新 release（v0.42.0）用於確認變更是否已穩定；既有 downstream 用於觀察慣例如何實際落地。

來源	地位	用途
模板 <code>main</code> 的 <code>init.sh</code>	權威	路徑清單、擁有權歸屬、符號連結目標
最新 release v0.42.0	參考	判斷某項變更是否已釋出。已比對： 兩者產生的 downstream 佈局完全相同 ，差異集中在 CI 工作流程與內部函式庫
既有 downstream	參考	觀察慣例的實際落地方式。五個抽查對象中 僅一個在 v0.42.0 ，其餘停在 v0.12.1 至 v0.41.0——其符號連結仍指向舊路徑 <code>.base/script/</code> 、 <code>setup.conf</code> 仍在 <code>config/docker/</code> 。這些是版本落差，不是替代慣例

以 `main` 為準的必要配套：釘住參照點

「main」不是可驗證的參照——它每天都在變。半年後若要回答「當初對齊的是什麼」，指向 `main` 沒有意義。

因此對應的 ADR 必須記錄具體的 **commit 與日期**（撰寫本文件時參照的是 2026-09-02 的 `main`）。模板有新變更時，是一次明確的、可被 review 的對齊動作，而非默默跟隨。

3.3.1 三種擁有權

downstream 的檔案分成三類，混淆會導致升級時被覆蓋或衝突：

類別	內容	升級時的行為
模板擁有 (符號連結)	<code>justfile</code> 、 <code>script/{build,run,exec,stop,prune,setup,setup_tui}.sh</code> 、 <code>script/justfile</code> 、 <code>script/{docker,base,template}/</code>	指向 <code>.base/dist/</code> , 隨升級自動更新。 不可修改
專案擁有 (真實檔案)	<code>Dockerfile</code> 、 <code>.setup.conf</code> 、 <code>config/</code> 、 <code>script/entrypoint.sh</code> 、 <code>script/hooks/</code> 、 <code>script/local/</code> 、 <code>script/</code> 下的專案腳本、應用程式碼、 <code>test/</code> 、 <code>doc/</code>	種子檔由模板初次產生後即歸專案， 升級不覆蓋
操作者擁有	<code>.setup.conf.local</code> 、 <code>.env.local</code> (未納入版控)	永不被改寫

3.3.2 目錄結構

<code>config-manager/</code>	
├─ <code>Dockerfile</code>	根目錄。多階段： <code>sys</code> → <code>devel-base</code> → <code>devel</code> → <code>runtime</code> → <code>runtime-test</code>
├─ <code>.setup.conf</code>	根目錄。服務定義來源；模板據此 + 主機偵測 產生 <code>.env</code> 與 <code>compose.yaml</code>
├─ <code>.setup.conf.local</code>	操作者覆蓋 (未納入版控)
├─ <code>.hadolint.yaml</code> <code>.dockerignore</code> <code>.gitignore</code>	
├─ <code>justfile</code>	→ <code>script/justfile</code> (模板擁有)
├─ <code>config/</code>	專案自有的設定資產
│ └─ <code>pip/requirements.txt</code>	
│ └─ <code>shell/bashrc</code>	
├─ <code>script/</code>	
│ └─ <code>build.sh</code> <code>run.sh</code> <code>exec.sh</code> <code>stop.sh</code> <code>prune.sh</code>	→ <code>.base/dist/...</code> (模板)
│ └─ <code>setup.sh</code> <code>setup_tui.sh</code>	→ <code>.base/dist/...</code> (模板)
│ └─ <code>justfile</code> <code>base/</code> <code>docker/</code> <code>template/</code>	→ <code>.base/dist/...</code> (模板)
│ └─ <code>entrypoint.sh</code>	專案自有。容器啟動點
│ └─ <code>hooks/</code>	專案自有的擴充點
│ │ └─ <code>pre/{build,run,exec,stop,prune,setup,setup_tui}.sh</code>	
│ │ └─ <code>post/</code> (同上七支)	
│ └─ <code>local/</code>	專案自有的 <code>just</code> 命令組
│ │ └─ <code>justfile.local</code>	註冊點
│ │ └─ <code>cfg/justfile.cfg</code>	<code>just</code> <code>cfg</code> <code>import</code> / <code>scan</code> / <code>apply</code> / <code>revert</code>
└─ <code>ci/</code>	專案自有的 CI 腳本
├─ <code>src/</code>	應用程式碼與資產，全部需測試覆蓋
│ └─ <code>core/</code>	純邏輯，不做 I/O
│ │ └─ <code>config_list.py</code>	清單檔載入 / 完整性檢查 / 寫回
│ │ └─ <code>state.py</code>	四種狀態判定
│ │ └─ <code>validate.py</code>	三層驗證
│ │ └─ <code>whitelist.py</code>	路徑白名單的讀寫與判定
│ │ └─ <code>identity.py</code>	名稱推導與 <code>uid</code> 產生
│ │ └─ <code>inference.py</code>	型別推斷與 <code>schema</code> 骨架
│ │ └─ <code>index.py</code>	參數索引與搜尋
│ │ └─ <code>attributes.py</code>	<code>config</code> 屬性與群組樹
│ │ └─ <code>roles.py</code>	角色權限對照
│ │ └─ <code>drafts.py</code>	草稿與進版
│ │ └─ <code>models.py</code>	<code>pydantic</code> 資料模型
│ └─ <code>io/</code>	外部互動，測試時可替換為 <code>fake</code>
│ │ └─ <code>parsers/</code>	<code>yaml</code> / <code>json</code> / <code>toml</code> / <code>ini</code> / <code>raw</code>
│ │ └─ <code>git.py</code>	<code>subprocess</code> 包裝
│ │ └─ <code>writer.py</code>	原子寫出 + <code>chmod/chown</code>
│ └─ <code>api/</code>	
│ │ └─ <code>routes/</code>	HTTP 端點
│ │ └─ <code>session.py</code>	使用者身分與單一編輯階段 (時鐘由外部注入)
│ │ └─ <code>cli.py</code>	CLI 入口 (API client, 見 §3.5)
└─ <code>web/</code>	前端
│ └─ <code>config_manager.html</code>	唯一入口頁
│ └─ <code>static/{js,css}/</code>	
├─ <code>test/</code>	
│ └─ <code>pytest/{unit,integration,system,acceptance}/</code>	
│ └─ <code>bats/smoke/</code>	建置期驗證 (容器起得來、端點有回應)
│ └─ <code>lint/{ruff,mypy,pylint,hadolint}/</code>	
│ └─ <code>fixtures/</code>	測試用 <code>config</code> 樣本與假 <code>repo</code>
│ └─ <code>reserved/</code>	非功能性型別佔位
└─ <code>doc/</code>	
│ └─ <code>adr/</code>	NNNNNNNN-<slug>.md
│ └─ <code>changelog/</code>	
│ └─ <code>test/</code>	測試說明
└─ <code>CONTEXT.md</code>	領域詞彙表

幾點取自實際 downstream 的觀察：**應用程式碼的位置模板不規定** (參照專案把 Python 套件放在自訂的頂層目錄，`src/` 反而放資產)；**web 資產獨立成頂層目錄**；`doc/` 下有 `adr/`、`changelog/`、`test/` 的固定分區。

script/ 底下要不要分子目錄

依數量決定，不預設。模板不強制，參照專案的做法是少量腳本直接放 script/ 根目錄（其 ci/ 則因腳本成組而分了子目錄）。

本專案的判準：同一類腳本累積到三支以上就分子目錄。過早分目錄會讓每支腳本都要多一層路徑卻換不到可讀性；過晚分則會讓 script/ 根目錄變成一堆看不出關聯的檔案。目前規劃 ci/ 一組，其餘（如一次性的初始化腳本）先放根目錄，超過門檻再收攏。

3.3.3 啟動腳本

script/entrypoint.sh 是專案自有的真實檔案，不是符號連結。模板只規定它的位置與一個契約：最後必須以 exec "\$@" 交棒，使服務成為 PID 1 的實際承載者，SIGTERM 才送得到、容器才停得掉。內容多寡由專案決定。

參照專案的兩個極端都存在：Isaac Sim 專案的只有一行 exec "\$@" ；Al agent 專案的則做了啟動內部 daemon、複製憑證、解密環境變數等一連串準備。

本專案的 entrypoint 做兩件事後交棒：

職責	說明
前置契約檢查	確認 config-repo 掛載存在且為有效 git repo、清單檔可解析。失敗即以非零碼退出——對應不變式 2：契約在做實事之前先自我驗證，不帶著壞掉的狀態把服務啟起來
exec 交棒	exec "\$@"。實際指令由 .setup.conf 定義：backend 為 API 伺服器，frontend 為靜態檔案伺服器

frontend 與 backend 共用同一份 entrypoint.sh，它不含服務專屬邏輯。日誌落地與存活監看由模板的執行期腳本提供（位於 .base/ 內），專案不重造。

3.3.4 Hooks 與命令組

擴充點	本專案的用途
script/hooks/pre/run.sh	啟動前檢查 config-repo 掛載點在主機端存在，不存在則中止並說明——錯誤發生在離現場更近的地方，而非等容器起來才發現
script/hooks/post/build.sh	建置後執行 smoke 測試
script/local/cfg/justfile.cfg	本專案的 CLI 命令組。註冊後可用 just cfg import <path>、just cfg scan、just cfg apply <uid> ——每一支都是 § 3.5 的 API client

為何 CLI 走 script/local 而非另立入口

script/local/ 是模板留給專案註冊自有命令組的地方，且明確標示為「repo 擁有、升級不覆蓋」。走這條路，本專案的 CLI 自動獲得與 just docker build 一致的命令形式與參數規則（由大到小、以 --option 收窄），使用者不需要學第二套慣例。另立入口是重造一個競爭版本的任務系統，導入模板時要拆掉。

3.3.5 模組必須可獨立使用

要求

每個模組的輸入與輸出都必須明確到可以單獨呼叫、單獨測試。不允許「要先跑過另一個模組才能用」這種隱含順序，也不允許透過共用的可變全域狀態互相傳話。

規則	具體要求
輸入明確	所有輸入來自參數。不從環境變數、不從全域設定物件、不從「上一步剛好寫進去的」共用狀態讀取
輸出明確	回傳結果或拋出具名例外。不以修改傳入物件作為主要輸出，不以「設定某個旗標讓下游去讀」的方式傳遞
無隱含順序	模組可任意順序呼叫。若 B 需要 A 的結果，那個結果必須是 A 的回傳值並由呼叫者傳給 B，不是 A 留在某處讓 B 自己去撿
無隱藏 I/O	core/ 的任何函式都不碰檔案系統與 git。需要檔案內容就把內容當參數傳進去（原則 N-4）

模組	輸入	輸出
core/config_list	清單檔的原始文字	已驗證的資料模型；或具名例外（重號、目標重複、來源不存在、路徑逃逸）
core/state	一筆條目 + 目標內容雜湊 + 來源內容雜湊 + 目標是否存在	四種狀態之一。 不讀檔 ，雜湊由呼叫者算好傳入
core/validate	內容文字 + 格式 + 規則集	問題清單（檔案、行號、欄位、修正建議）。空清單代表通過
core/whitelist	白名單規則 + 待判定路徑	允許／拒絕 + 拒絕原因
core/identity	目標路徑 / 匯入時刻	推導出的名稱 / uid
core/inference	解析後的資料	欄位路徑對型別的對照；schema 骨架
core/index	uid + 資料 / 查詢字串 + 範圍	展平的 (uid, 參數路徑, 值) 三元組 / 命中清單
core/attributes	清單檔 + 屬性異動	更新後的清單檔；群組樹結構
core/roles	角色 + 動作	允許或拒絕
core/drafts	草稿集合 + 進版請求	待寫出的變更清單；或驗證問題（指出是哪一份的哪一項）
api/session	使用者 + 目前時刻（外部注入）	階段或已被占用（含持有者資訊）。 不讀系統時間 ——否則逾時行為無法測試
io/parsers	文字 + 格式	資料結構 + 原樣資訊（註解、順序）；或解析錯誤含行號
io/git	操作 + 參數	結構化結果；或含 git 原始輸出的例外
io/writer	目標路徑 + 內容 + 權限	成功或例外。 要嘛完整寫入、要嘛完全不動

可驗證的判準：core/ 的每支測試都能在沒有檔案系統、沒有 git、沒有網路的情況下跑完。做不到就代表該模組有隱藏依賴，是設計問題不是測試問題。

3.3.6 測試目錄的一處分歧

參照專案與模板慣例不一致，本專案採模板慣例

模板的測試佈局慣例是**工具優先**（test/<工具>/<層級>/）。參照專案的 bats 測試遵循此慣例（test/bats/smoke/），但其 pytest 測試採了層級優先（test/unit/pytest/、test/integration/pytest/）——同一個 repo 內兩種形狀並存。

本專案統一採**工具優先**（test/pytest/unit/、test/bats/smoke/）。理由：與模板的 dispatcher 分派邏輯一致，導入時不需遷移；而參照專案的 pytest 佈局看起來是隨手加入的漂移，不是刻意的替代方案。詳見 § 3.6.2。

模組依賴

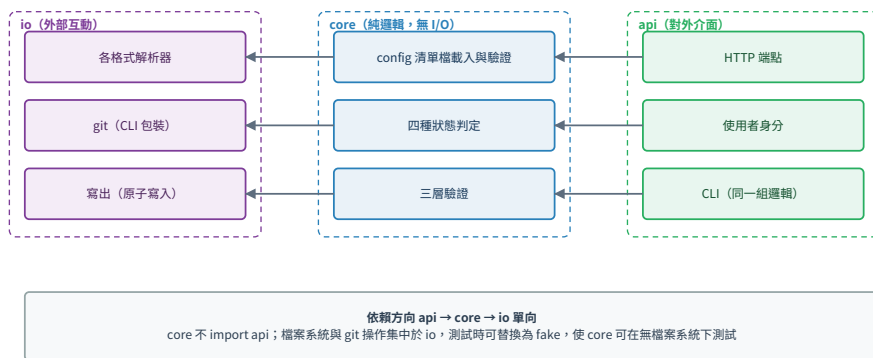


圖 4 | 後端模組結構與依賴方向

依賴方向為 api → core → io 單向

core/ 不 import api/；所有檔案系統與 git 操作集中於 io/，測試時可替換為 fake，使 core/ 的狀態判定與驗證邏輯能在無檔案系統的情況下測試。這是原則 N-4，也是整個系統可測性的地基。

3.4 config-repo 目錄結構

```

config-repo/
├── config-list.toml      ← 單一真實來源：納管清單 + apply 依據
├── .schemas/            ← JSON Schema (v0.8.0 起)
├── .rules/              ← semantic lint 規則 (v0.8.0 起)
├── .lintrc.toml          ← 正規形式規則的開關 (v0.8.0 起, 見 §6.4)
├── files/               ← config 實際內容, 以 group 分目錄
│   ├── amr01/
│   │   ├── nav2_params.yaml
│   │   └── lidar_merger.yaml
│   └── system/
│       └── etc_docker_daemon.json

```

files/ 以 group 為第一層目錄，**不鏡射原始絕對路徑**：原始路徑橫跨 /etc、/opt、/home，鏡射後目錄樹過深，且檔案搬移時需連動修改。目標路徑由 config 清單檔記錄，同名衝突以 group 目錄隔離。

JSON Schema 驗證的是「解析後的資料」，不是「JSON 檔」

名字容易誤導。JSON Schema 描述的是**資料結構**——物件、陣列、字串、數字、布林——而 YAML、TOML、INI 解析之後得到的**就是同一套資料結構**。因此同一份 schema 可以驗證任何格式的 config：



所有格式一律經過同一道把關，沒有例外。`.schemas/` 目錄名沿用 JSON Schema 這個既有標準的稱呼，不代表只適用於 JSON 檔。

唯一不受此把關的是 `format = "raw"` 的項目——那是刻意宣告「不解析、只做版控」的檔案（例如二進位或無法解析的自訂格式）。這類項目在介面上**明確標示為「未驗證」**，使用者看得到自己放棄了什麼，而不是以為有在把關（不變式 2）。

`.rules/`（跨欄位規則）同理，作用於解析後的資料，與原始格式無關。

3.5 介面設計

3.5.1 CLI 與介面共用同一條路徑

原則 N-5 要求每個介面操作都有 CLI 對等。達成方式有兩種，差別很大：

做法	機制	評估
共用函式庫	CLI 與 HTTP 端點各自呼叫同一組 <code>core/</code> 函式	只保證 編譯期 共用。兩邊各自的參數處理、預設值、錯誤轉換仍會分歧，且分歧不會被任何測試抓到
CLI 作為 API client	CLI 透過 HTTP 呼叫與介面完全相同的端點	採用 。保證 執行期 同一條路徑——不存在「CLI 能做但介面不能」或反之的情況，因為根本是同一組端點

這是業界的既有做法，不是新設計：

專案	形態
Docker	<code>docker</code> 指令是 <code>daemon</code> HTTP API 的 client。沒有「本機直呼」的捷徑路徑
Kubernetes	<code>kubectl</code> 與 Web Dashboard 都是 <code>apiserver</code> 的 client，走同一組 REST 端點
GitHub CLI	<code>gh</code> 是 GitHub API 的純 client，與網頁介面共用同一套 API

此形態即 Hexagonal Architecture（Ports and Adapters）：`core/` 是 port，HTTP 是唯一的 adapter，CLI 與瀏覽器都是該 adapter 的使用者。

代價與為何可接受

CLI 需要 backend 服務在跑。Docker 也是如此，這是可接受的常態。

本專案有一個特殊考量：**服務掛掉時要能修 config**。但這裡不需要 CLI 特例——`config-repo` 就是一個普通 git repo，任何時候都能直接用 `git` 操作與手動編輯。救援路徑已經存在，不需要在 CLI 開一條會分歧的旁路。

3.5.2 API 風格：資源走 REST，動作走子資源

本系統的操作分成兩類：清單與單筆查詢是資源導向，而寫出、退版、處置偏離是**動作**。純 REST 表達不了後者，硬套會產生形式上是 REST、語意上是 RPC 的端點。

採用 Kubernetes API 慣例的解法：其 API Conventions 明確規定，當資源需要暴露與該資源緊密耦合的替代動作或視圖時，應以新的子資源（sub-resource）表達。Kubernetes 自身即以此提供 `/status`、`/scale`、`/exec`、`/log`、`/binding` 等子資源——`/scale` 甚至是虛擬資源，不對應 `etcd` 中任何儲存物件，純粹作為動作介面存在。

同一模式在其他大型 API 上也是主流：GitHub REST API 以 `PUT /repos/{owner}/{repo}/pulls/{n}/merge` 表達合併動作、Stripe 以 `POST /v1/payment_intents/{id}/capture` 表達請款。三者都不是把動作硬扭成資源的 CRUD，而是承認動作的存在並給它一個子路徑。

結論

資源查詢走標準 REST，狀態變更走 `POST /api/configs/{uid}/<動作>` 子資源。端點語意誠實，且與 Kubernetes、GitHub、Stripe 的既有慣例一致——這對未來要接自動化的人是零學習成本。

3.5.3 端點

方法	路徑	說明
POST	/api/session	設定使用者身分（姓名、email）
GET	/api/configs	列出所有納管項目，含狀態。可帶 <code>?group=</code> 、 <code>?hostname=</code>
POST	/api/configs	納管新檔案
GET	/api/configs/{uid}	取得單筆的 metadata 與內容
PUT	/api/configs/{uid}	修改內容（驗證 → 記錄 → 寫出，原子）
DELETE	/api/configs/{uid}	解除納管（目標檔案保留）
GET	/api/configs/{uid}/history	變更歷史。可帶 <code>?prefix=cfg,adopt</code>
POST	/api/configs/{uid}/apply	動作：寫出至目標位置
POST	/api/configs/{uid}/revert	動作：退回指定版本
POST	/api/configs/{uid}/resolve	動作：處置偏離（ <code>overwrite</code> 或 <code>adopt</code> ）
POST	/api/validate	僅驗證不儲存，供編輯時即時回饋
POST	/api/scan	執行全項目差異掃描
GET	/api/browse?path=	檔案系統瀏覽（受白名單限制），供納管使用

錯誤回應必須結構化

驗證失敗不可只回字串訊息。回應需包含可定位的欄位——檔案、行號、欄位名、修正建議——供介面直接在編輯器上標示。這是對應不變式 2 的介面層要求：錯誤要能被機器處理，才能被準確地呈現給人。

3.6 測試架構

依不變式 7，測試門檻的**承諾**是不變式，具體的分類法與覆蓋率工具是可置換的機制。本節定義本專案採用的分類法——與組織模板的分類法一致（附錄 A.1），但工具依語言而異（pytest 而非 bats、pytest-cov 而非 kcov）。

3.6.1 三個正交軸

過去常見的錯誤是把三個不同性質的東西混成一個「四分類」：Unit 與 Integration 是**層級**（範圍），Smoke 是**型別**（目的），Lint 是**靜態分析**（根本不是動態測試）。本專案明確分成三軸。

軸 1 | 靜態分析

不是動態測試層級。位於 `test/lint/<tool>/`。本專案的工具：`ruff`、`mypy`、`pylint`、`hadolint`（Dockerfile）。

軸 2 | 層級（範圍階梯）

層級	範圍	本專案的具體對象
Unit	單一函式／模組，隔離狀態下	<code>core/</code> 的狀態判定、驗證、config 清單檔完整性（N-4 使此層可完全無 I/O）
Integration	數個模組協作	納管流程、commit + apply 的原子性、round-trip 讀寫、git wrapper 對真實 repo 的操作
System	整個建好的映像，端到端	容器啟動 → API 可用 → 完整使用週期（納管／修改／退版／偵測）
Acceptance	使用者實際收到的東西與其 UX	Web UI 的可操作性、錯誤訊息可行動性、CLI 的 help 與命令表面

最高層級是 **System**。「End-to-end」是在 System／Acceptance 層執行的**型別**，不是層級名稱。

軸 3 | 型別（目的，套用於某個層級）

- **Smoke** — 建置驗證：「它到底起不起得來」的關鍵子集，在 `runtime-test` stage 的建置過程中執行。
- **End-to-end** — 一條從頭到尾的完整流程。
- **Regression** — 守住某個已修好的缺陷。**每個修好的 bug 都應留下一條 regression 測試。**
- **保留（非功能性）** — Performance／Security／Usability／Reliability，先建空目錄佔位，需要時再填。

3.6.2 目錄結構

```
test/
├── pytest/
│   ├── unit/           # core/ 的隔離測試，無檔案系統依賴
│   ├── integration/    # 對真實 git repo 與檔案系統
│   ├── system/         # 對建好的容器
│   └── acceptance/     # UI / CLI 的使用者視角
├── lint/
│   ├── ruff/
│   ├── mypy/
│   ├── pylint/
│   └── hadolint/
├── fixtures/           # 測試用的 config 樣本、假 repo
├── reserved/           # 非功能性型別的佔位 (.gitkeep)
│   ├── performance/
│   ├── security/
│   ├── usability/
│   └── reliability/
```

未使用的欄位保留空目錄 + `.gitkeep`，使結構本身完整且自我說明——不需讀本章也看得出框架有哪些格子。

3.6.3 測試檔鏡射原始碼

規則

測試檔鏡射原始碼檔。原始碼的結構由設計決定，絕不由測試檔的數量或形狀決定。

- 一個原始碼檔預設對應一個 spec，名稱對應：`core/config_list.py` ↔ `test/pytest/unit/test_config_list.py`。
- 確實需要多個子單元 spec 的模組，開一個以原始碼檔命名的資料夾：`test/pytest/unit/config` 清單檔/`test_<subunit>.py`。資料夾的**存在**本身就是「這個模組有多個測試單元」的訊號；單一 spec 的模組保持扁平，不過度結構化。
- 絕不為了達成 1:1 對應而拆分原始碼檔（見 § 0.2 N-4）。
- Integration / end-to-end 測試**放在它所驅動的 **orchestrator** 旁，**不放在葉節點模組旁**。一個驅動整條流程的測試（例如 apply：驗證 → 寫入 → commit → 寫出）屬於 orchestrator 的 spec，因為那正是它測的東西。只有真正隔離的單元測試才移到葉節點模組的 spec。

3.6.4 命令模型

`just` 命令模型：**零特例，每個動作都是一個 namespace**，沒有任何東西在 **top-level**。自建版本的內容自己寫，但命名與選項規則現在就對齊（附錄 A.1）。

```
just docker build          # 預設 stage
just docker build --stage runtime # 指定 stage
just docker run / exec / stop

just test                  # 全部：lint + 所有層級 + coverage
just test --file <path>   # 單一 spec 檔
just test --filter <regex> # 符合樣式的 spec
just test --level unit    # 單一層級
just test lint            # 所有 linter
just test lint --ruff     # 單一 linter
```

兩條規則：

- lint 不是 test 的同級**，它是 `just test lint`——lint 是測試的一部分。新開 top-level 命令需要自己的 justfile 與腳本，不值得。
- 由大到小：每個命令裸跑是最大範圍**，透過 `--option` 收窄，絕不用意義取決於位置的裸位置參數。

3.7 測試項目定義

實作採測試先行，因此測試介面必須先議定

測試先行的規則是：**沒有經過確認的測試介面，不寫測試**。在寫任何測試之前，先寫下要測的介面並取得確認——因為測不了全部，事先議定測試介面是讓測試力氣落在關鍵路徑與複雜邏輯上、而不是散在每個邊界案例的方法。

完整清單見隨附文件包 `doc/TEST-PLAN.md`（十八個測試介面、六個驗收旅程、覆蓋率審計、明確不測的項目）。該文件目前狀態為提案，需逐項確認後才進入實作。

3.7.1 什麼是「測試介面」

測試介面是「從哪個位置觀察行為」。同一段程式碼可以從很多位置觀察，但只有公開介面這個位置是穩定的。

觀察位置	測試長什麼樣	好壞
內部實作	「有沒有呼叫 <code>chmod</code> ？」「暫存檔名對不對？」	壞。改寫實作就壞，但行為沒變
公開介面	「呼叫 <code>write()</code> 之後，目標檔案的內容與權限對不對？」	好。內部怎麼改都不影響

這與 § 3.3.5 的模組介面契約是同一件事的兩面——那裡定義的輸入輸出，就是這裡要觀察的位置。

編號 `T1 - T18` 為單元／整合／系統層，`A1 - A6` 為驗收層。

3.7.2 測試介面一覽

代號	測試介面	層級	驗證的核心行為
T1	清單檔載入與寫回	Unit	六種完整性錯誤各自產生具名例外；寫回時清單檔本身的註解與順序保留
T2	狀態判定	Unit	四種狀態的判定與順序。 刻意不讀檔 ，摘要值由呼叫者傳入
T3	驗證	Unit	三層各自的問題偵測；每個問題含檔案、行號／欄位、修正建議；同一份 schema 驗三種格式結果一致
T4	白名單判定	Unit	允許／拒絕； <code>../</code> 與符號連結逃逸皆被攔截
T5	身分推導	Unit	名稱推導規則；uid 連續呼叫嚴格遞增
T12	型別推斷與人工指定	Unit	浮點與整數區分正確 （ROS 型別錯誤的來源）；人工指定優先於推斷；清除後回到推斷值且逐欄位獨立
T13	編輯階段生命週期	Unit	取得／續期／逾時回收；開發與部署模式的差異。 必須注入時鐘 ——讀系統時間的實作無法有效測試
T14	參數索引與搜尋	Unit	巢狀展平為參數路徑； 修改後舊值不再命中、解除納管後索引移除 ——最容易漏的兩處；搜尋範圍指定時排除其他範圍的命中
T16	config 屬性與群組	Unit	改群組後樹重建；多群組時在每個節點下都出現；父節點彙總最嚴重狀態
T18	草稿與進版	Unit	儲存草稿不記錄也不寫出；任一草稿驗證失敗則整批不進版；寫出失敗時整批回滾；草稿與四種狀態正交
T17	角色權限	Unit	兩種角色的權限對照正確。 驗證的是對照表本身，不驗證「無法偽造角色」 ——v0.10.0 前沒有那個保證，也不假裝有
T6	解析與原樣寫回	Integration	未改動時逐位元組相同 ；改一個值後註解與格式全保留
T7	變更紀錄	Integration	類型過濾正確；退版後歷史未被截斷。 透過本測試介面驗證，不直接跑 git 指令檢查
T8	原子寫出	Integration	權限正確； 寫入中斷時目標維持原內容
T15	啟動前置檢查	Integration	掛載缺失、非 git repo、清單檔不可解析皆 以非零碼結束且不啟動服務
T9	HTTP 端點	System	結構化錯誤回應；驗證失敗不留紀錄也不寫出；編輯階段衝突的回覆內容
T10	CLI	System	只驗證參數映射到端點正確 ，不重複驗證業務邏輯
T11	瀏覽器端到端	System	使用者看得到的行為。 以語意屬性選取元素，不以 CSS class 或 DOM 路徑

3.7.3 驗收層

系統層與驗收層間的不是同一個問題

層級	問的問題
系統層	這個功能運作正確嗎？（按鈕停用、項目出現在樹上、狀態判定正確）
驗收層	使用者能達成他的目的嗎？

差別具體在哪：**系統層的每一步都通過，但步驟順序讓人卡住時，系統層測不出來。驗收測試以完整的使用者旅程撰寫，且用使用者的語言描述目標，不用內部術語。**

代號	旅程	失敗判準
A1	把一份新 config 納入管理	任一步需要離開介面 （改設定檔、下指令、問人）即為不通過。含被白名單擋下後從錯誤訊息直接加入的路徑
A2	調整一個參數並確認生效	需要知道 YAML 語法才能修改；或改完後無法確認改了什麼、誰改的
A3	發現並處置一次偏離	需主動掃描才看得到警示；或兩個處置選項的後果在按下之前沒說清楚
A4	出錯後能自行修正	使用者看完錯誤訊息後仍不知道下一步該做什麼 。任何只說「格式錯誤」或「操作失敗」的訊息即為不通過
A6	開發者整理一份新納管的 config	任何開發者專屬功能在一般使用者模式下「顯示但停用」 ——那會讓人以為權限不足，實際只是沒切換角色
A5	未受訓者可自行操作	無法自動化 。由未參與開發的同事依序完成 A1–A4，另一位開發者完成 A6，全程不得詢問操作方式。v0.9.0 的驗收條件

A1–A4 與 A6 可腳本化，A5 不行——**能不能自己摸索出來，只能由真的沒摸索過的人來測。**

3.7.4 覆蓋率審計

測試介面清單必須**完整**——每個模組與每條核心流程都要落在至少一個測試介面上。以下為對照結果，完整表格見 `doc/TEST-PLAN.md`。

流程	單元層	整合層	系統層	驗收層
納管	T4 → T5 → T12 → T14 → T1	T6 → T7	T9 → T11	A1
修改與進版	T3 → T14	T6 → T7 → T8	T9 → T11	A2、A4
退版	—	T7 → T8	T9 → T11	A2
偏離偵測	T2	T6 → T8	T9 → T11	A3
偏離處置	T3（納入路徑必經）	T7 → T8	T9 → T11	A3
解除納管	T1 → T14	T7	T9 → T11	—
白名單維護	T4、T17	T1	T9 → T11	A1
屬性與群組	T16、T17	T1 → T7	T9 → T11	A6
型別指定	T12、T3、T17	T7	T9 → T11	A6
容器啟動	—	T15	—	—
編輯階段	T13	—	T9 → T11	—

三個刻意的空格

沒有測試介面	為何可以沒有
資料模型	是資料宣告，本身無行為。其約束（必填、型別）在 T1 載入時被驗證
模式判定（開發／部署）	讀設定回傳一個列舉值，無邏輯可測。其 效果 在 T13（逾時差異）與 T11（介面差異）被驗證
三層驗證的「層」本身	層是組織方式不是行為。各層的實際檢查在 T3 逐項驗證

刻意留空與漏掉的差別在於**有沒有寫下理由**。沒有理由的空格就是漏掉。

3.7.5 明確不測的

不測	理由
前端 DOM 結構、CSS class、元素排列順序	改樣式即壞而行為沒變。 介面行為改由 T11 以語意選取器驗證 ，不是不測
第三方函式庫的內部行為	不是我們的行為
HTTP 框架的路由機制	同上
核心層的私有函式	反模式：測實作細節，重構就壞
「某函式有被呼叫」	反模式：以呼叫次數或順序斷言

3.7.6 三個反模式

反模式	辨識方式與後果
耦合實作	模擬內部協作者、測私有函式、或繞過介面從側面驗證（例如直接跑 <code>git log</code> 檢查而非呼叫 <code>history()</code> ）。徵兆： 重構時測試壞了，但行為沒變
套套邏輯	期望值以與實作相同的方式重算，因此由構造保證通過，永遠不會與程式碼相左。 期望值必須來自獨立來源 ：已知的良好字面值、手算的範例、或規格
橫向切片	先寫完全部測試再寫全部實作。這會驗證 想像中的行為 ——測到的是形狀而非使用者在意的行為，且在理解實作之前就把測試結構定死

3.7.7 迴圈規則

- **紅燈先行**。先寫會失敗的測試，再寫剛好讓它通過的程式碼。不預先為未來的測試寫東西，不加投機性功能。
- **一次一片**。一個測試介面、一個測試、一個最小實作。
- **垂直切片**。每個測試是一發曳光彈，依上一輪學到的東西調整下一輪。
- **重構不在迴圈裡**，它屬於 review 階段。

與 § 8 版本里程碑的關係

§ 8.2 的驗收檢查點是**結果**（此版本結束時使用者能做什麼），本節的測試介面是**觀察位置**，兩者不同層次。

一個版本的實作是**多輪垂直切片**——每輪在某個測試介面上加一個測試、寫最小實作——直到該版本的驗收檢查點通過。**不是「先寫完該版本的所有測試再實作」**，那正是橫向切片。

4. 資料格式選型與 config 清單檔規格

4.1 兩個層次的格式問題

層次	是否可選	說明
被管理的 config 格式	不可選	由既有系統決定：ROS 2 參數為 YAML、docker daemon.json 為 JSON、部分工具鍵為 TOML。系統 必須全部支援 ，這是相容性需求而非選型問題。
config 清單檔本身的格式	可選	本系統自訂的檔案，需擇一。

4.2 config 清單檔格式比較

判斷前提：使用者透過 HTML 介面閱讀解析後的內容，不直接閱讀 raw 檔案。此前提使「註解支援」與「人工閱讀友善度」的權重大幅下降，剩下的唯一標準是**嚴謹度**——即同一份檔案被不同 parser 讀取時，是否可能得到不同結果。

面向	TOML	JSON	YAML
型別明確性	最高。Integer／Float／Boolean／String／Datetime 各有獨立文法記號	高。但 number 不區分 int／float	最低。裸字面的解析依 YAML 版本而異
解析歧義	無。規格明確，實作間差異極小	無	多。no／08／1.10 等（見 §6.3）
縮排語意	無關	無關	有。縮排錯誤造成靜默失效
註解支援	有	無	有
深層巢狀	笨拙	簡潔	簡潔
Round-trip parser	tomlkit	原生	ruamel.yaml

結論：config 清單檔採用 TOML

- **排除 YAML**：解析歧義最多、有縮排語意。config 清單檔是整個系統的地基，其解析結果不應有任何不確定性。
- **排除 JSON**：型別明確性足夠，但完全不支援註解。config 清單檔雖主要由程式寫入，維護者仍需要在檔案內留下區塊性說明（例如「以下為 v2 案場專用」），這類資訊不適合塞進每個條目的 description 欄位。
- **選 TOML**：型別文法最嚴謹、無縮排語意、支援註解。原先反對 TOML 的理由是「深層巢狀笨拙」，但本 config 清單檔結構僅兩層，permissions 以 inline table 撰寫即可，該缺點在此結構下不成立。

前提的邊界

「使用者不看 raw data」在 code review 與現場緊急處理時並不成立——git diff 檢視的就是 raw 內容。此前提不影響 TOML 的選擇（其 diff 可讀性本就良好），但在後續設計時不應據此假設 raw 檔案永遠不會被人閱讀。

4.3 config-list.toml 規格

身分設計：三個欄位，三種職責

身分不用單一欄位承擔，因為「機器要用的」與「人要看的」是兩種相反的需求：前者必須永不改變，後者必須隨時可改。合成一個欄位等於兩邊都做不到。此設計與 Kubernetes 的 Name／UID 分離一致：其設計文件將 UID 定義為系統產生的不透明值，保證在時間與空間上唯一，用來區分歷史上相似的實體；Name 則由客戶端提供、鼓勵對人友善，在特定範圍內唯一，用於資源網址與跨操作的引用。物件刪除後，同樣的名字可以再次使用，UID 則不會重複。

欄位	誰產生	會不會變	用途
uid	系統，納管時產生一次	永不變	唯一的真實識別碼。所有內部關聯（歷史查詢、狀態關聯、commit 前綴的比對）只認它
name	系統從路徑推導，人可改	可改	純粹給人看。就算過期或命名不一致也不影響任何功能
hostname	納管時讀系統 hostname，人可改	可改	實例維度。name 相同、hostname 不同者即為同一份 config 的不同實例

hostname 是納管時寫入的值，不是執行期讀取的值

系統在納管當下讀取本機 hostname 作為預設值填入，之後**永不再讀**。人不需要手打，但值由檔案記錄——這正是不變式 8 所要求的形式。

若改為執行期解析會壞在四處：hostname 因重新配置或映像複製而改變時，config 的身分會靜默改變；同一映像開出的多台機器在改名前 hostname 相同，直接撞號；repo 被 clone 到開發機時讀到的是開發機的名字，紀錄全部對不上；未來要在 A 機器管理 B 機器的 config 也無法實作。

完整參照形式

```
<name>@<hostname>-<uid>
navigation-params@amr01-mfz3k9q1
```

編號在最後，前面兩段是給人讀的裝飾。commit 訊息維持可讀，同時可用 -mfz3k9q1 精確搜尋整段歷史：

```
cfg(navigation-params@amr01-mfz3k9q1): 調整 max_vel_x 至 0.8
```

name@hostname 的模板／實例形式取自 systemd 的 unit 命名（getty@tty1.service）：同一個邏輯單元跑在不同位置，共用名字，以 @ 後段區分。這使跨機比對可直接

以 `name` 分組——若不同機器各拿一個無關的識別碼，那個功能就無從實作。

各欄位的產生規則

欄位	規則
uid	由匯入時刻的毫秒時間戳轉 base36 產生（8 碼），寫入後永不變更。唯一性來自時間單調遞增——backend 是唯一寫入者，不存在競爭，因此不需碰撞檢查。附帶效果：uid 可排序，納管順序被保留下來。用毫秒而非秒是因為批次納管會在同一秒內完成；用 base36 而非可讀時間字串是因為後者 17 碼，出現在每筆變更紀錄裡太長
name	由目標路徑最後二至三層推導，去除副檔名、底線轉連字號、去重疊： /opt/robot/navigation/params.yaml → navigation-params /etc/docker/daemon.json → docker-daemon 納管後可自由修改，因為它不是 key
hostname	納管時讀取本機 hostname。可修改（例如改為場域代號而非機器名）

檔案範例

```
list_version = 1

# 未個別指定時套用的預設權限
[defaults.permissions]
owner = "root"
group = "root"
mode = "0644"           # 字串，避免被解析為整數

[[files]]
uid      = "mfz3k9q1"           # 匯入時間戳，永不變
name     = "navigation-params"  # 給人看，可改
hostname = "amr01"             # 納管時讀入，可改
source   = "files/amr01/nav2_params.yaml"  # repo 內相對路徑
target   = "/opt/robot/config/nav2_params.yaml"  # 目標絕對路徑
format   = "yaml"              # 明寫，不由副檔名推斷
groups   = ["navigation"]      # 多對多 tag
description = "Nav2 導航參數"
schema   = ".schemas/nav2.json" # 選填，v0.8.0 起

[[files]]
uid      = "mfz3k9r7"
name     = "docker-daemon"
hostname = "amr01"
source   = "files/system/etc__docker__daemon.json"
target   = "/etc/docker/daemon.json"
format   = "json"
groups   = ["system"]
requires_privilege = true      # 需提權寫入
permissions = { owner = "root", group = "root", mode = "0644" }  # inline table
```

欄位	必填	說明
uid	是	系統產生的唯一識別碼，永不變更。所有內部關聯以此為準
name	是	人可讀名稱，非 key，可自由修改
hostname	是	實例維度。納管時自動填入，可修改
source	是	Repo 內相對路徑
target	是	部署目標絕對路徑
format	是	yaml／json／toml／ini／raw。明寫而非由副檔名推斷（.yaml、.param 等變體不可靠）。raw 表示不解析，僅做版控
groups	否	分群 tag，可多個。機器維度不放這裡，已由 hostname 承擔
description	否	用途說明，於介面顯示
schema	否	對應 JSON Schema 路徑（v0.8.0 起）
permissions	否	覆寫 defaults。Git 僅記錄 executable bit，完整 mode／owner／group 須在此保存
requires_privilege	否	標記需提權寫入的目標。介面顯著標示，寫出走 sudoers 白名單路徑

載入時的完整性檢查

- uid 唯一 — 重複則拒絕載入。
- (name, hostname) 組合唯一 — 非強制正確性所需（uid 已保證唯一），但重複會使顯示形式產生歧義，應警告。
- target 唯一 — 兩筆指向同一絕對路徑時，最終內容取決於寫出順序，屬靜默 bug，必須攔截。
- source 檔案存在 — config 清單檔有紀錄但檔案不存在應報錯。
- target 為絕對路徑且不含 .. — 路徑逃逸防護。
- format 為已知值。

5. 核心流程

5.1 流程 A | 新增 config (納管)

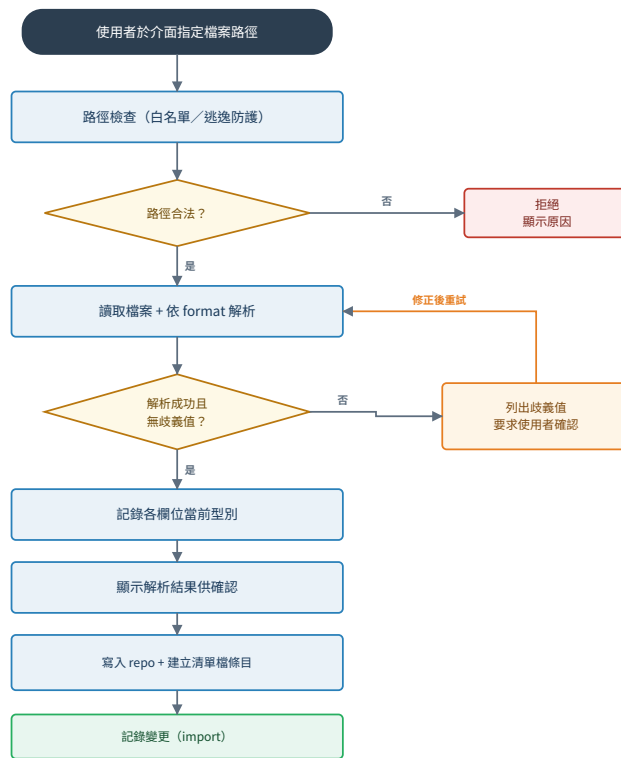


圖 5 | 納管流程

流程說明。納管的目的是把一份已存在於磁碟上的 config 納入版控，過程中**不改變檔案內容**。使用者從檔案瀏覽器選定路徑後，系統先做路徑檢查——確認落在白名單目錄內、不含 `..` 逃逸、且為可讀的一般檔案；不合法直接拒絕並回報具體原因，不進入後續步驟。

通過後依 config 清單檔指定的 `format` 解析檔案。解析階段有兩種失敗：語法錯誤（檔案本身壞掉）直接拒絕；含有 § 6.3 所列歧義值（例如 YAML 的 `no`）則**不拒絕，而是列出清單要求使用者確認其意圖**。這是刻意的設計——既有檔案幾乎必然含有這類寫法，若在納管階段硬擋將無法導入。使用者確認後，該筆判定結果記入 config 清單檔，後續修改時即以此為準。

「顯示解析結果供確認」在 v0.2.0 的範圍

此步驟拆成兩層，只有前者在 v0.10.0 前實作：

- **歧義值警告清單 (v0.2.0, 必要)** ——檔案含 `no`、`08`、`1.10` 等寫法時列出並要求確認意圖。這是不變式 2 的硬要求：靜默匯入一個可能被誤解析的值，就是靜默失敗。無法延後。
- **完整匯入檢視 (v0.10.0 之後, 見 附錄 B.3)** ——逐欄位型別對照、解析後的結構樹、與原檔並排比對、批次匯入預覽。這些讓人「看得更清楚」，不是「防止出錯」，可以延後。

v0.2.0 的最小版本為：歧義值清單 + 一行摘要（欄位數、偵測到的格式、原始權限）。

解析成功後系統推斷各欄位的當前型別（`string/int/float/bool/list/dict`）並記錄，作為日後修改時的型別一致性依據。v0.2.0 僅做到這一層，不產生完整 JSON Schema。

最後將檔案複製至 `files/<group>/`、讀取原始檔案的 `owner/group/mode` 寫入 config 清單檔條目，並以 `import(<id>)` 前綴 commit。此時 **target 位置的原始檔案完全未被觸碰**，系統狀態為一致。

5.2 流程 B | 修改與進版

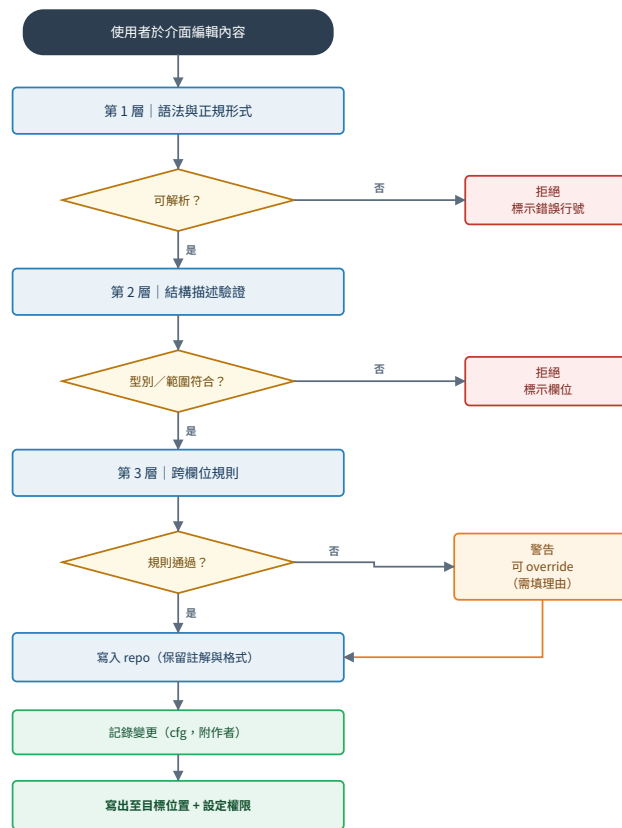


圖 6 | 修改與進版流程

流程說明。使用者在編輯畫面修改內容後按下儲存，內容依序通過三層驗證。第 1 層 檢查能否解析，以及所有值是否符合正規形式白名單（§ 6.3）；第 2 層 依 config 清單檔指定的 schema 檢查型別與數值範圍；第 3 層 執行跨欄位的 semantic 規則。前兩層失敗直接拒絕 commit 並標示錯誤行號或欄位，第三層失敗則顯示警告並允許使用者填寫理由後 override。

驗證通過後以 round-trip parser 寫回 `files/`，**保留原檔的註解、引號樣式與鍵順序**——僅改動使用者實際修改的部分。接著以 `cfg(<uid>)` 前綴 commit，author 為使用者在身分輸入頁填寫的 name 與 email。

最後執行寫出：以「寫入暫存檔 → fsync → 設定 owner/group/mode → rename」的順序寫出目標位置。rename 在同一 filesystem 內為原子操作，確保讀取者不會看到半殘檔案。**commit 與 apply 必須成對發生**——任一失敗即整體回滾，不留下「已 commit 未 apply」的中間狀態，這是 § 5.4 差異偵測能正確判讀的前提。

各版本的驗證範圍

圖 6 呈現完整三層。v0.3.0 僅實作 第 1 層，第 2 層／3 於 v0.8.0 加入。未實作的層級直接通過，不阻擋流程。

5.3 流程 C | 退版

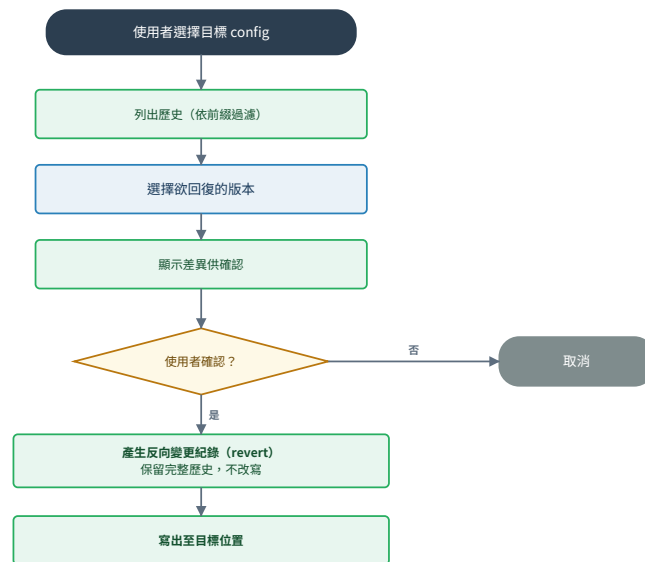


圖 7 | 退版流程

流程說明。使用者選定一份 config 後，系統列出其 commit 歷史。歷史預設以前綴過濾（僅顯示 `cfg` 與 `adopt`，即實際改變內容的變更），避免退版產生的 `revert` commit 與 metadata 變更的 `meta` commit 干擾判讀。使用者可切換為顯示全部。

選定目標版本後，系統顯示該版本與當前版本的 diff 供確認。確認後執行 `git revert` ——產生一個新的 commit 來抵銷後續改動，而非移動 branch pointer。這意味著歷史完整保留：可以看到「曾經改成 A、又改成 B、然後退回 A」的完整軌跡。若使用 `reset --hard`，中間的 commit 會從歷史消失，而那正是稽核最需要的資訊。

`revert` commit 以 `revert(<顯示形式>): rollback to <sha>` 為訊息，最後執行與流程 B 相同的寫出目標位置。

5.4 流程 D | 偵測 config 差異

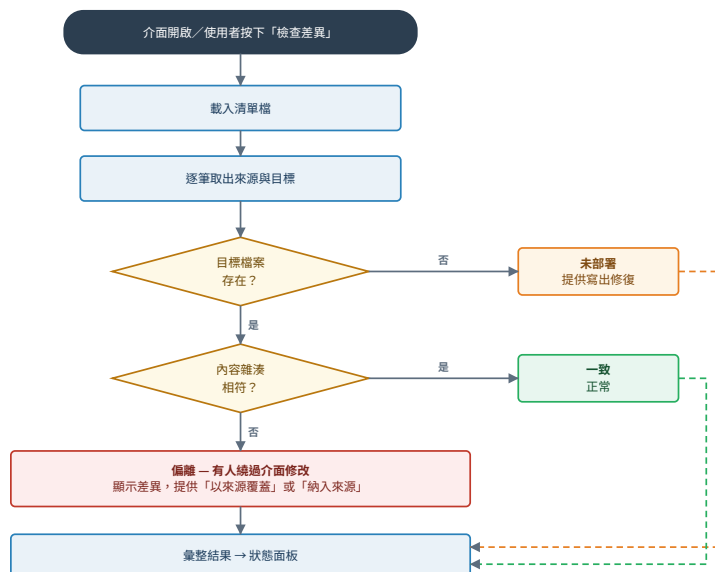


圖 8 | 差異偵測流程

流程說明。差異偵測在兩個時機執行：使用者開啟主畫面時自動執行一次，以及使用者主動按下工具列的「檢查差異」按鈕。**v0.10.0 不做定期背景掃描**——手動觸發已足以涵蓋日常使用，背景排程與通知列為未來規劃（附錄 B.3）。

系統載入 config 清單檔後逐筆比對。先確認 `target` 檔案是否存在，不存在即為 `未部署`（可能被誤刪或尚未 `apply`），提供一鍵寫出修復。存在則計算內容 hash 與 `repo` 中對應檔案比對，相符為一致，不符為偏離。

偏離的語意之所以明確，來自 § 5.2 的原子性保證。因為介面內的每次修改都是「commit 與 `apply` 成對發生」，系統不存在合法的「`repo` 已改但 `target` 未更新」狀態。因此任何不一致，必然代表有人繞過介面直接修改了 `target`。UI 對此提供兩種處置：以 `repo` 覆蓋 `target`（放棄該修改），或將 `target` 現況納回 `repo`（承認該修改，以 `adopt` 前綴 commit）。

5.5 Config 狀態轉換

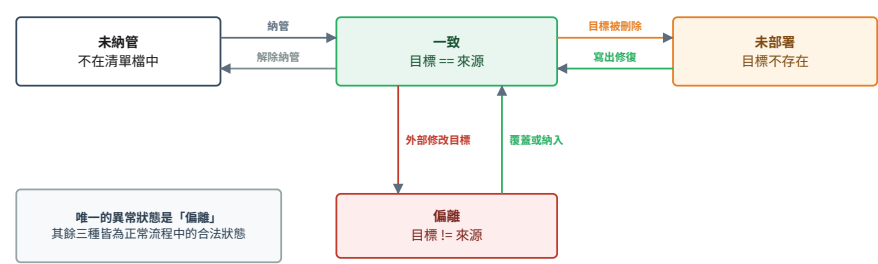


圖 9 | Config 狀態機

說明。系統中每一份 config 恆處於四種狀態之一。UNMANAGED 是初始狀態——檔案存在於磁碟但不在 config 清單檔中，系統對其一無所知。納管後進入一致，這是正常運作下應該恆常維持的狀態。

偏離 與 未部署 都是偏離狀態，但性質不同：**未部署 是可預期的**（檔案被刪、或新環境尚未 apply），一鍵修復即可；**偏離 是異常訊號**，代表流程被繞過，需要人為判斷該修改是否合理，因此提供兩種相反的處置而非單一修復動作。

解除納管會回到 UNMANAGED，但**不刪除 target 檔案**——解除管理不應改變系統當前行為，只是系統不再追蹤它。歷史內容仍在 git 中，需要時可翻查。

狀態	判定條件	UI 呈現與可用動作
未納管	不在 config 清單檔中	於納管精靈的檔案瀏覽器中可見，可選取納管
一致	target == repo	綠色標籤。可編輯、檢視歷史、退版、解除納管
偏離	目標 ≠ 來源	紅色標籤 + 整列底色。 點擊進入差異檢視，提供覆蓋／納入兩種處置
未部署	target 不存在	橘色標籤。提供 寫出修復

6. 驗證機制與值的正規形式

6.1 為何 Web 介面優於直接編輯

Web 介面本身不提供任何保證，價值來自其作為驗證與稽核的掛載點：

1. **統一入口** — 提供 gate 的掛載位置。此為必要條件而非充分條件，仍需搭配差異偵測確認流程被遵循。
2. **後端驗證為 gate** — 可直接拒絕不合格的修改。編輯器的 LSP 僅提供建議，仍可存檔。
3. **Form view 消除結構性錯誤** (v0.8.0) — 由 schema 生成表單，數值欄位為 number input、列舉為 dropdown，使用者在結構上沒有機會輸入錯誤型別或縮排錯誤。
4. **自動產生 commit 紀錄** — 修改者、時間、差異自動留存。

6.2 三層 Lint

層級	工具	攔截範圍	處置	導入版本
Syntax	ruamel.yaml/tomlkit/json/configobj	能否解析、正規形式白名單 (§ 6.3)、重複 key	硬擋	v0.3.0
Schema	jsonschema/ajv	型別、必填欄位、拼錯的 key、數值範圍	硬擋	v0.8.0
Semantic	自訂規則	跨欄位約束 (min_vel < max_vel 、 inflation_radius > robot_radius)、外部一致性 (topic/frame 是否存在)	警告 + 可 override	v0.8.0

Semantic 層的重要性

型別正確但數值不合理的 config，比型別錯誤的 config 危險得多。型別錯誤導致 node 啟動失敗，問題立即暴露；inflation_radius 設定過小則要到車輛碰撞才會發現。

Override 機制

Semantic 層不宜硬擋。現場異常狀況下若因規則誤判無法修改設定，介面會成為路障。做法：警告 + 強制填寫 override 理由 + 於 commit message 標記。Override 紀錄本身即有價值——同一規則被反覆 override 代表規則需修正。

6.3 值的正規形式白名單（目前版本）

原則：白名單之外一律報錯，不做靜默修正

系統定義一組明確接受的字面形式。不在表列範圍內的值**直接報錯並要求使用者明確化**，不做任何自動轉換。自動將 no 改為 "no"，若原意確為布林值則屬改壞配置且無人知曉，比不處理更危險。

類別	接受	拒絕（報錯）	理由
Boolean	true false	yes no on off True TRUE y n	YAML 1.1 將 yes/no/on/off 解析為布林、1.2 不會。國家代碼 NO 被誤判為 false (Norway problem)。若意圖為字串必須加引號： "no"。
Integer	0 123 -5	08 0o17 0x1F 1_000 +5	前導零在 YAML 1.1 為八進位、1.2 為十進位，跨版本不一致。底線分隔與進位前綴增加解析歧義。
Float	0.0 1.5 -2.5	1e3 .5 1. inf nan .inf	科學記號在部分 parser 回傳字串。省略整數或小數部分的寫法解析行為不一致。特殊值於數值運算中行為難預期。
Float 尾數	—	1.10 (警告)	解析為 1.1，尾隨零遺失。若原意為版本號應改用字串 "1.10"。
ROS 2 double	1.0	1	宣告為 double 的參數若寫整數，觸發 InvalidParameterTypeException，node 無法啟動。已知型別時強制要求小數點。
String	加引號 "text" 純文字（無歧義時）	與其他型別字面衝突的裸字串	裸字串僅在不與布林／數值／null 字面衝突時接受。
Null	null	~ Null NULL 空值 (key:)	~ 可讀性差。空值易與「忘記填」混淆。
縮排	2 個空白	Tab 字元 空白與 Tab 混用 非 2 的倍數	Tab 在 YAML 規格中不允許作縮排。混用造成解析結果與視覺不符。
尾隨空白	無	行尾有空白	肉眼不可見，但會成為字串值的一部分，導致比對失敗。
重複 key	—	同層級重複 → 一律報錯	YAML 規格未定義行為，多數 parser 靜默取最後一筆，造成「明明寫了卻沒生效」的難查問題。 錯誤訊息須明確列出重複的 key 名稱及其出現的所有行號 ，不得僅回報「發現重複」。

Parser 設定

- 明確指定 **YAML 1.2**（`ruamel.yaml` 的 `typ='rt'`），迴避 1.1 的布林值陷阱。
- 納管時顯示每個值的解析型別供使用者確認。
- Tab 縮排、尾隨空白於介面上以可視化標記顯示。

既有檔案的過渡處理

現有 config 極可能包含表列的拒絕形式。納管階段**標示為警告並列出清單**，允許使用者確認後以現況匯入（記錄於 commit message）；後續透過介面修改時才套用硬擋規則。如此既不阻礙導入，又能逐步收斂至正規形式。

6.4 規則可配置化（v0.8.0）

§ 6.3 是目前版本的規則，不是永久定義

不同專案、不同格式對嚴謹度的需求不同。v0.8.0 起將白名單抽出為 `config-repo/.lintrc.toml`，隨 config 一同版控——退版時規則一併回退，避免以新規則驗舊內容產生假警報。

```
# config-repo/.lintrc.toml (v0.8.0 規劃)
[yaml]
yaml_version = "1.2"
boolean_literals = ["true", "false"]      # 白名單，其餘報錯
allow_octal_prefix = false
allow_scientific_notation = false
duplicate_key = "error"                    # error | warn (不提供 ignore)
trailing_whitespace = "error"
indent = { style = "space", width = 2 }

[yaml.overrides]
# 個別檔案豁免，需填寫理由
"files/legacy/old_params.yaml" = { boolean_literals = "any",
                                     reason = "第三方產生，暫不修改" }

[toml]
# TOML 文法本身已足夠嚴謹，僅檢查重複 key
duplicate_key = "error"
```

設計約束： `duplicate_key` 不提供 `ignore` 選項。重複 key 的靜默失效是本系統要解決的核心問題之一，允許關閉會使該保證失效。個別檔案的豁免須透過 `overrides` 並填寫理由，使豁免本身成為可稽核的紀錄。

7. UI／UX 設計

7.1 設計原則

1. **狀態優先**。主畫面第一眼要回答「有沒有東西不對勁」，而非「有哪些檔案」。
2. **不可逆操作需二次確認**。退版、以來源覆蓋目標、解除納管三項，皆先顯示將發生的實際變更。
3. **錯誤訊息必須可行動**。不寫「格式錯誤」，寫「第 7 行 no 不在正規形式白名單內；布林請寫 false，字串請加引號」。
4. **驗證未通過時儲存按鈕停用**，不讓使用者按下去才被拒絕。
5. **所有需要調整的動作都能在介面完成**，包含白名單維護（§ 7.9）。要求使用者記指令等於把系統的可用性押在記憶力上。
6. **CLI 對等**。每個介面動作都有對應的 CLI 指令（原則 N-5）。
7. **Wireframe 是規格，不是插圖**。本章的圖由 tools/ 的腳本產生，實作介面時對照 doc/UI-ELEMENTS.md ——該文件把 wireframe、HTML 元素、測試選取器綁在一起。三者不同步時，測試會通過但介面是錯的。

7.2 單一入口，單一編輯階段

7.2.1 頁面結構：一頁到底

只有一個 HTML 檔：`config_manager.html`

不多做頁面切換。所有視圖（清單、編輯、歷史、差異、白名單）都在同一頁內以區塊顯示或切換。

理由	說明
狀態不需重建	多頁面每次切換都要重新取得清單與狀態。單頁只在開啟與手動觸發時掃描一次
編輯脈絡不中斷	使用者常需要「一邊看差異一邊改」。跨頁面做不到，同頁分區塊很自然
單一編輯鎖易於維護	多頁面時「使用者是切換頁面還是開了第二個分頁」難以分辨，鎖的判定會變複雜
與部署形態一致	frontend 容器只供應靜態資源。一個檔案的部署面積最小

7.2.2 一次只允許一個編輯階段

本系統不支援多人同時編輯，也不支援同一人開多個分頁編輯。

機制	說明
取得階段	頁面載入時向 backend 要求編輯階段。成功則取得階段識別碼，並定期送出續期
已被占用	backend 回覆已有活躍階段，含 持有者姓名 、 email 、 開始時間 。前端顯示唯讀模式並明確說明是誰、從何時開始持有
釋放	正常關閉頁面時主動釋放；異常中斷則靠續期逾時自動釋放
接管	不提供強制接管。持有者逾時後階段自然釋放，其他人再取得

為何不提供強制接管

強制接管會產生「兩個人都以為自己在編輯」的窗口，正在編輯的內容可能靜默遺失——這是不變式 2 要避免的失效類型。逾時釋放雖然要等，但過程明確：持有者知道自己逾時了，接手者知道前一位已離開。

7.2.3 兩個正交的概念：角色與環境

這是兩件事，不要混為一談

概念	誰決定	影響什麼
角色	使用者在身分輸入時 自行選擇	能做什麼 ：修改參數型別、維護白名單、編輯 config 屬性（群組）——這三項僅開發者可用
環境	backend 依部署階段判定，前端無法變更	階段行為 ：是否提供「記住此裝置」、是否閒置逾時

兩者必須正交：**部署環境下仍可能需要開發者調整白名單**，若把角色綁在環境上，那件事就做不到。

7.2.3.1 角色

角色	可做的事
一般使用者	檢視、修改參數值、退版、處置偏離、納管（在既有白名單內）
開發者 ↘	上述全部，另加： 修改參數型別 （§ 7.5.4）、 維護白名單 （§ 7.9）、 編輯 config 屬性 （§ 7.4.3）

v0.10.0 前為自我宣告——身分輸入頁上一個明顯的切換，沒有驗證。這與身分輸入本身的性質一致（§ 7.8）：目的是讓使用者清楚知道自己現在處於哪個模式，而不是阻擋。

介面上僅開發者可用的元素一律加上 ↘ 標記，且**一般使用者模式下直接不顯示**，而非顯示後停用——後者會讓人以為自己權限不足，實際上只是沒切換角色。

之後若需要真正的權限控管，應在反向代理層處理，並改為由 backend 回報角色，不在本系統實作認證。

7.2.3.2 環境

行為	開發模式	部署模式
「記住此裝置」	提供	不顯示。現場機器可能多人共用，記住身分會導致變更被歸到錯的人
閒置逾時	不逾時	10 分鐘無操作自動退出，同時釋放編輯階段
逾時前提示	—	剩餘 1 分鐘時顯示提示，可延長
模式來源	由 backend 依 <code>.setup.conf</code> 的部署階段判定並回報給前端。不由前端自行判斷——前端的判斷可被繞過	

7.3 導航模型

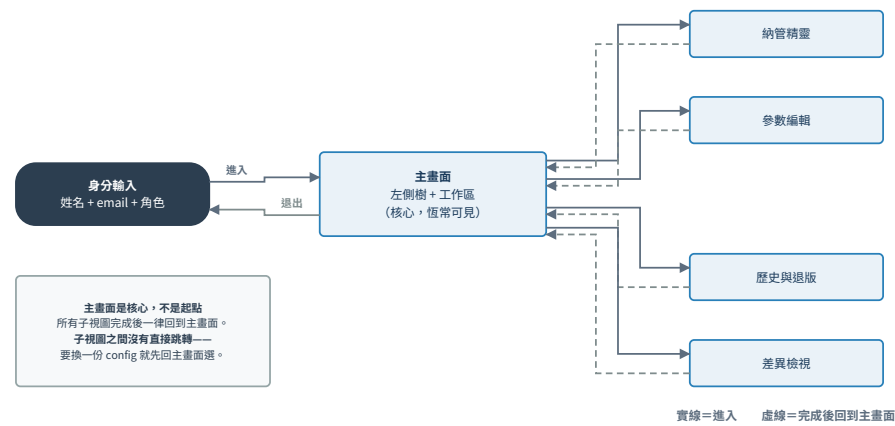


圖 10 | 單頁內的視圖切換

主畫面是核心，不是起點

所有子視圖完成後一律回到主畫面。這不是頁面跳轉——依 § 7.2.1 的決定，全部在同一頁內以區塊切換，主畫面的左側樹恆常可見。

規則	說明
雙向	每個子視圖都有明確的返回路徑。完成動作後自動返回，未完成時可取消返回
子視圖之間沒有直接跳轉	要換一份 config 就先回主畫面選。不做「從編輯畫面直接跳到另一份的歷史」——那會讓「我現在在看哪一份」變得難以追蹤
返回時保留位置	樹的展開狀態、捲動位置、已展開的區塊都維持原狀（§ 7.10）
返回不丟草稿	進入子視圖再返回，未進版的草稿仍在。只有退出與階段逾時會清除草稿（§ 7.4.3）
退出是唯一的出口	主畫面 → 身分輸入。有未進版草稿時二次確認

為何扁平而不做多層導航：使用頻率極不平均——日常操作有九成落在「查看狀態」與「修改某份 config」。複雜的導航層級只會增加點擊成本，而深層路徑也會讓「我現在在哪」更難判斷。

7.4 主畫面：左側樹 + 右側工作區

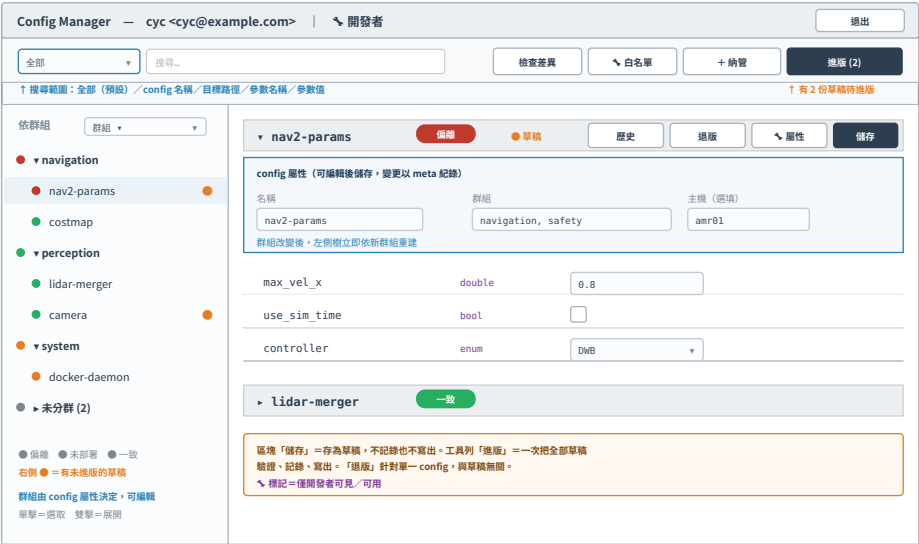


圖 11 | 主畫面：左側樹 + 右側工作區

7.4.1 左側樹以群組為階層

樹的階層來自 **config** 的群組屬性，不是機器。本系統管理的是單一主機上的 config，因此不存在跨機器的階層。

▼ navigation nav2-params costmap ▼ perception lidar-merger camera ▼ system docker-daemon ▶ 未分群 (2)
--

行為	說明
階層來源	config 的 groups 屬性。使用者編輯屬性後，樹立即依新群組重建
多重群組	一份 config 可屬多個群組，會出現在多個節點下 (tag 而非樹狀分類)
未分群	沒有 groups 的項目集中在此節點，提示使用者去分類
主機 (選填)	可切換為以 hostname 為第一層。現階段非必要——單機部署下所有項目的 hostname 相同，分層沒有意義。保留此切換是為了未來多機時不需改動結構
單擊 / 雙擊	單擊選取；雙擊展開參數至右側工作區
同時展開多個	右側以可折疊區塊堆疊，各自獨立儲存，便於跨檔比對參數
狀態標示	節點左側顯示狀態色點；父節點彙總子節點的最嚴重狀態 (偏離 > 未部署 > 一致)
草稿標示	節點右側橘點表示有未進版的草稿。與狀態色點分開，見 § 7.4.3

7.4.2 搜尋：範圍可選

搜尋框左側有**範圍下拉選單**，預設「全部」，可指定後過濾：

範圍	範例輸入	結果
全部 (預設)	max_vel	以下所有範圍的聯集
config 名稱	nav2	名稱符合的 config 節點
目標路徑	/opt/robot	路徑符合的 config
參數名稱	max_vel	含此參數的所有 config，展開時定位到該列
參數值	0.55	任何參數為此值的 config

指定範圍的用途是**降噪**：搜尋 camera 在「全部」下會同時命中 config 名稱與參數名稱，指定範圍後只看想看的那類。

搜尋歷史 (v0.10.0 之後)：保留最近 10 筆，含當時選定的範圍，存於瀏覽器端。列為後續項目是因為需先確認搜尋語法穩定，否則存下的歷史會失效。

7.4.3 儲存與進版是兩件事

三段，不是兩段

編輯 → 儲存 (草稿) → 進版 (記錄 + 寫出, 原子)

區塊上的「儲存」只是存為草稿——不產生變更紀錄、不寫到目標位置。要一次改三份 config 時，三個區塊各自儲存，最後按一次進版。

若兩者綁在一起，改三份 config 就產生三次版本迭代——而它們在語意上是同一次調整。歷史會被切碎，退版時也無法一次退回那批改動之前。

動作	位置	行為
儲存	各區塊	存為草稿。跑第 1 層驗證 (語法與正規形式)，不通過則不存。草稿只在編輯階段內存在
進版	工具列 (全域)	把 全部 草稿一次驗證、記錄、寫出。按鈕上顯示待進版的草稿數
捨棄變更	工具列 / 各區塊	清除草稿，回到來源內容。全域或單一 config
退版	各區塊	與草稿無關 。針對單一 config 回到先前版本，走 § 7.6 的歷史流程
歷史	各區塊	檢視該 config 的變更紀錄

進版時的原子性

規則	說明
全部驗證才進版	任一草稿的任一層驗證失敗， 整批不進版 ，並指出是哪一份的哪一個參數。不做「先進通過的那幾份」
每份一筆紀錄	同一次進版產生多筆變更紀錄 (每份 config 一筆)，但在同一次操作中完成。這維持 § 2.3 的「範圍放 uid」規則，且「這份 config 改過幾次」仍然準確
記錄與寫出仍原子	單一 config 的記錄與寫出成對發生 (不變式 2)。 草稿層在那之前，不影響此保證
失敗即整批回滾	寫出過程中任一失敗，已寫出的回復、已產生的紀錄一併撤銷

草稿不是第五種狀態

四種狀態（一致／偏離／未部署／未納管）比較的是**目標與來源**。草稿存在編輯階段，不在來源也不在目標，因此**與狀態正交**——一份 config 可以是「一致」而且「有草稿」。

介面上以獨立指示呈現：樹節點右側一個橘點表示有未進版的草稿，與左側的狀態色點分開。**混成同一個標示會讓「檔案不一致」和「我還沒送出」看起來像同一件事**，而它們的處置完全不同。

草稿存在哪裡

選項	評估
編輯階段（採用）	存在 backend 的階段狀態中。重新整理頁面不遺失；階段逾時釋放時一併清除並提示
瀏覽器端	重新整理即遺失，且無法在階段逾時時給出明確提示
git 工作區（未提交變更）	不可 。會讓來源 repo 的工作區與 HEAD 不一致，偏離偵測就多出一種無法分辨的狀態

7.4.4 編輯 config 屬性

群組不是自動產生的，需要使用者指定。**屬性面板可從工作區的區塊標題展開**：

欄位	誰可改	說明
名稱	開發者	納管時由路徑推導，可改。純顯示用，改名不影響任何關聯（\$ 4.3）
群組	開發者	可多個。 儲存後左側樹立即依新群組重建
主機	開發者	納管時自動填入，可改為場域代號。選填
說明	開發者	用途說明，於樹節點滑鼠停留時顯示

屬性變更以 `meta` 類型記錄（\$ 2.3），與內容變更分開，因此歷史檢視的預設過濾不會被屬性調整淹沒。

為何屬性編輯限開發者

群組決定整個面板的組織方式。若一般使用者可改，現場調參時誤改群組會讓其他人下次開啟時找不到熟悉的位置——這不是資料損毀，但會造成混淆而且難以察覺。

7.5 參數編輯：欄位表，不是文字編輯器

這不是線上 YAML 編輯器

使用者**不編輯檔案文字**。介面呈現的是一列一個參數的欄位表，每個參數依型別給對應的輸入控制項——概念上等同 ROS 的參數調整介面。

若讓使用者在網頁的文字框裡輸入 YAML，那與用 vim 沒有區別，前面所有關於型別安全與結構驗證的設計都失去意義：使用者一樣要自己處理縮排、引號與型別。

nav2-params | 參數編輯 | 開發者模式

/opt/robot/config/nav2_params.yaml

群組 navigation, safety | 權限 root:root 0644 | 型別來源：推斷 + 3 項人工指定

一致

參數	型別	值	來源值	驗證
max_vel_x	double	0.8	0.5	✓
max_accel	int	3	3	✓
use_sim_time	bool	☐	false	✓
controller	enum	DWB	DWB	✓
robot_frame	string	base_link	base_link	✓
inflation_r	double	0.15	0.55	✗ 小於 robot_radius 0.3
recovery_behaviors	list	+ 新增 [0] spin ↑ ↓ ✗ 移除 [1] backup ↑ ↓ ✗ 移除 [2] wait ↑ ↓ ✗ 移除 ↑ ↓ 交換順序		
costmap (異狀物件)	object	內部邏輯套用同一套規則		

「儲存」＝存為草稿；工具列的「進版」才會記錄與寫出 | 開發者模式才能修改型別欄；一般使用者只看得到型別，不能改人工指定的型別存入 schema，可個別清除回到推斷值

取消

儲存

圖 12 | 參數欄位表

7.5.1 欄位表的一列包含什麼

欄位	說明
參數名稱	巢狀以縮排或可折疊區塊表示，不用點號串成長字串
型別標示	<code>double</code> / <code>int</code> / <code>bool</code> / <code>string</code> / <code>list</code> 。型別由系統決定，使用者不能改
輸入控制項	依型別給定，見 § 7.5.2。不用拉霸——拉霸適合即時視覺回饋的場景，config 調參需要的是精確輸入
目前值 / 來源值	已修改未儲存時並列顯示，改動一眼可見
驗證狀態	該列的驗證結果。錯誤時該列標紅並顯示原因與修正建議
說明	來自 schema 的 <code>description</code> ，滑鼠停留顯示

7.5.2 各型態的控制項

型別	控制項	保證與行為
<code>bool</code>	核取方塊	結構上不可能輸入非布林值
<code>int</code>	數字輸入框（整數步進）	不可能輸入小數或文字
<code>double</code>	數字輸入框（允許小數）	輸出一定帶小數點，直接消除 ROS 2 最常見的 <code>1</code> vs <code>1.0</code> 型別錯誤
<code>enum</code>	下拉選單	不可能輸入不存在的值。選項來自 schema
<code>string</code>	文字輸入框	引號由系統處理，使用者不需知道何時要加
<code>list</code>	可增刪的列 + ↑ ↓ 交換順序	元素型別一致。順序有意義（例如 <code>recovery</code> 行為的執行順序），因此除了新增與移除，必須能調整順序
巢狀物件	可折疊區塊	內部遞迴套用上述規則。縮排由系統產生，使用者不接觸

有數值範圍時（schema 提供，或未來由 ROS 參數描述取得），輸入框套用 `min` / `max` / `step`，超出範圍即時標示。

list 的順序調整不是可選項

只提供「新增／移除」的話，要把第三項移到第一位，使用者得刪掉重打——而重打會遺失原值、也容易打錯。順序本身就是資料，必須能直接操作。

7.5.3 修改參數型別 ↩

型別的來源有三層，後者覆蓋前者：

來源	誰產生	說明
推斷	系統	納管時依當前值推斷（§ 5.1）。預設值
人工指定 ↩	開發者	推斷錯誤時手動更正。會被記錄，可個別清除回到推斷值
參數描述	系統（未來）	從執行中的 ROS 節點取得（附錄 B.1）。有此來源時優先於推斷，但不覆蓋人工指定

行為	說明
誰能改	僅開發者。一般使用者看得到型別但不能改——欄位呈現為純文字而非下拉選單
記錄在哪	寫入該 config 對應的 schema 檔（ <code>.schemas/</code> ）。人工指定的型別就是 schema 的一部分，因此自動獲得版控、隨退版一併回退
視覺標示	已指定的列標示「已指定」與清除入口，與推斷值一眼可分
清除	移除該欄位的指定，回到推斷值。逐欄位清除，不是全部重設
變更紀錄	以 <code>meta</code> 類型記錄，與參數值的變更分開

為何需要人工指定

推斷只能看到當下的值。`timeout: 5` 會被推斷為 `int`，但作者的意圖可能是 `double`——這正是後續改成 `5.5` 時會出事的地方。推斷是起點不是答案，必須留一條人工更正的路。

反過來說，不可放給一般使用者：改錯型別會讓後續所有寫入都用錯誤的形式，而且直到 node 啟動失敗才會發現。

7.5.4 沒有 schema 時怎麼辦

v0.10.0 之前多數 config 沒有 schema，此時型別來自納管時的推斷結果（§ 5.1）：記錄各欄位當時的型別，編輯時以該型別給控制項。

已知型別的參數用欄位表，型別未知或結構複雜到無法表格化的，退回文字區塊並明確標示為「未結構化編輯」——使用者看得到自己失去了型別保護（不變式 2）。這類項目應優先補 schema。

7.5.5 元件庫評估

需求是參數表：型別對應控制項、巢狀折疊、陣列增刪、每欄位驗證狀態、目前值與來源值並列。以此對照候選：

候選	符合的部分	不符合的部分
Tweakpane lil-gui（同類）	參數面板的視覺語言正確；巢狀資料夾；無相依可直接引入；成熟穩定	沒有陣列綁定 （支援型別為數值、字串、布林、色彩、二至四維點；同類函式庫的作者亦指出這幾套都無法編輯陣列）； 沒有每欄位的驗證錯誤呈現 ；核心賣點是拉霸與即時監看，而本專案兩者都不需要
@json-editor/json-editor	由 schema 直接生成表單 ——本專案已有 schema； 處理陣列與巢狀物件 ；每欄位的驗證錯誤呈現；純 JavaScript 可直接引入，無建置流程	外觀較舊，需自訂樣式；僅支援 JSON Schema 較早的草案版本；維護節奏較慢
自建渲染器	完全符合。可精確控制「目前值 / 來源值 / 驗證狀態」三欄的呈現	需自行實作與維護，估計數百行；陣列與巢狀的邊界案例要自己處理

建議：以 @json-editor/json-editor 起步

它與本專案的需求對得最準——**schema 驅動**（我們本來就要產生 schema）、**支援陣列與巢狀**（config 一定有）、**有驗證錯誤的欄位級呈現**（不變式 2 的介面體現）、**且無建置流程**（符合單一 HTML 的決定）。

Tweakpane 不建議，儘管視覺語言最接近：缺陣列與驗證呈現這兩項，而它換來的拉霸與即時監看正是本專案不需要的。若採用，光是補陣列就要寫外掛，那不如自建。

自建作為備案：若「目前值 / 來源值」並列或差異標示無法在 json-editor 內做出來，改為自建。屆時仍可借用 Tweakpane 的資料夾與繫結模型作為結構參考。

7.6 歷史與退版

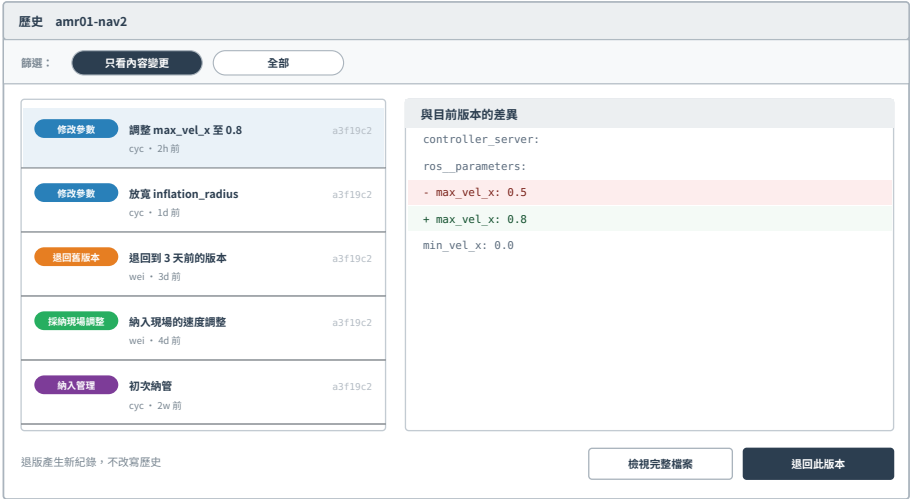


圖 13 | 歷史檢視與退版

左為變更列表、右為選定版本與當前版本的差異。

7.6.1 使用者看到的是行為，不是代號

介面上不出現 `cfg` / `revert` / `import` 這些字

那些是變更紀錄的內部類型（§ 2.3），存在的目的是讓歷史可以被程式過濾。使用者不需要知道它們，也不該被要求學。

內部類型	介面顯示	說明
<code>import</code>	納入管理	—
<code>cfg</code>	修改參數	—
<code>revert</code>	退回舊版本	附「退回到哪一版」的說明
<code>adopt</code>	採納現場調整	—
<code>meta</code>	調整設定	群組、名稱、型別指定等
<code>unmanage</code>	解除管理	—

篩選器同樣以行為描述呈現：「只看內容變更」（`cfg` + `adopt`，預設）／「全部」。

v0.10.0 之後：加入更細的顯示／隱藏切換，讓使用者自選要看哪幾類。v0.10.0 前只做兩檔，因為兩檔已能解決退版造成的歷史噪音問題，更細的分類在實際使用前無從得知哪些組合有用。

7.6.2 差異以參數為單位

顯示哪個參數從什麼值改成什麼值，與 § 7.5 的欄位表語言一致。**使用者不需要讀懂文字差異格式。**

7.7 差異檢視（偏離處置）

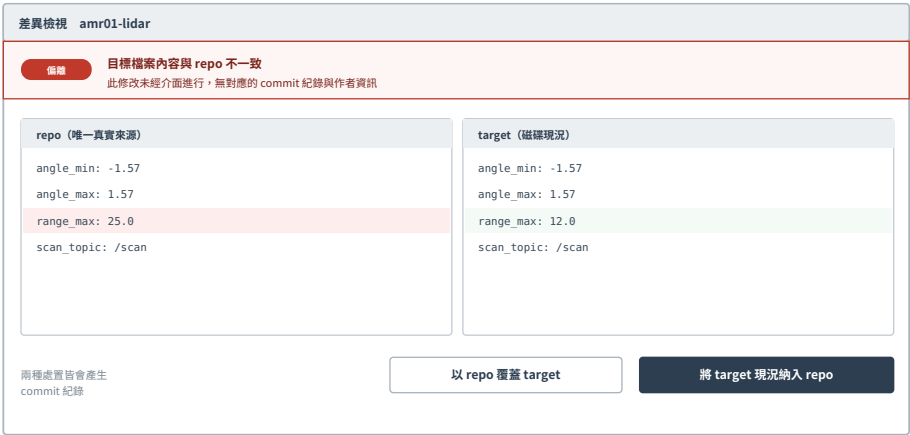


圖 14 | 差異檢視

僅在偏離狀態出現。頂部橫幅說明：目標檔案被修改，且該修改未經介面進行，因此沒有對應的變更紀錄與作者資訊。

主區域為左右並排的**參數比對**——左為來源、右為目標現況，差異參數標色。並排而非合併差異的理由：使用者在此要做的是「判斷哪一邊是對的」，並排更符合這個決策。

呈現方式與 § 7.6 的歷史差異一致——同樣以參數為單位、同樣的顏色語言。兩處是同一件事（比較兩個版本），介面不該長得不一樣。

7.7.1 顯示範圍切換（v0.10.0 之後）

模式	顯示什麼
只看差異（預設）	只列出兩邊不同的參數。config 有上百個參數而只差兩個時，這是唯一可用的檢視
完整檔案	列出全部參數，差異處標色。用於確認「其他地方真的沒被動過」

v0.10.0 前只做「只看差異」。原因是完整檢視在參數少時與只看差異幾乎相同，價值要到 config 變大之後才顯現，而屆時已有實際使用經驗可判斷該怎麼呈現。

處置	行為	適用情境
以來源覆蓋目標	執行寫出，目標內容被還原	該修改是誤操作或不該保留的臨時測試
將目標現況納入來源	讀取目標內容 → 驗證 → 記錄 → 寫出	該修改是有效的現場調整，需正式納入版控

兩者都產生變更紀錄，使偏離的解決過程本身也可追溯。「納入來源」同樣跑完整驗證——繞過介面的修改不因此獲得免驗證的特權。

7.8 身分輸入



圖 15 | 身分輸入頁

開啟時要求輸入姓名與 email。**這不是認證機制**——沒有密碼、沒有驗證。唯一目的是作為變更紀錄的作者，使變更可追溯到人。

介面文案採「請輸入身分資訊，將作為變更紀錄的作者」而非「登入」，避免使用者誤以為系統有存取控制。若未來需要真正的認證，應在反向代理層處理，不在本系統實作。

7.8.1 角色選擇

身分輸入頁上有一個**明顯的角色切換**：一般使用者 / 開發者。

設計	理由
明顯而非隱藏	目的是讓使用者清楚知道自己現在處於哪個模式，而非阻擋。隱藏起來會讓開發者以為功能不存在
v0.10.0 前不驗證	與身分輸入本身的性質一致——沒有認證機制，自我宣告。若需要真正的權限控管應在反向代理層處理
目前角色恆常可見	視窗標題列顯示  開發者 或不顯示。避免「以為自己是一般使用者卻改到了型別」
可隨時切換	不需重新輸入身分。切換為一般使用者時，僅開發者可用的元素 直接消失 而非停用
退出	主畫面標題列有 退出 按鈕，回到身分輸入頁並釋放編輯階段。 有未進版草稿時二次確認 ，說明草稿將遺失

「記住此裝置」僅在開發環境顯示（§ 7.2.3.2），且**不記住角色**——每次開啟都要重新選，避免下一個人沿用前一個人的角色。

7.9 白名單維護

納管時的路徑檢查依據一份白名單（§ 5.1）。**白名單的維護必須能在介面完成**——若要求使用者編輯設定檔或記指令，等於把系統可用性押在記憶力上（設計原則 5）。

 **僅開發者可修改**

白名單決定「系統可以碰主機上的哪些目錄」，是安全邊界。一般使用者可**檢視**目前的白名單（理解自己為何被擋），但**不能修改**。

納管被拒時的錯誤訊息對兩種角色不同：開發者看到「將此路徑加入白名單」的直接入口；一般使用者看到「此路徑不在允許範圍，請聯繫開發者加入」——**後者仍然說清楚下一步是什麼**，只是那一步不由他執行。

動作	介面行為
檢視	列出目前允許的路徑前綴，以及各自是誰、何時加入的
新增 	輸入路徑 → 系統即時顯示「此路徑下目前有幾個可納管檔案」供確認 → 送出
移除 	若已有納管項目落在該前綴下， 顯示受影響清單並要求確認 ，不靜默移除
納管被拒時	 開發者：錯誤訊息附「加入白名單」的直接入口。一般使用者：訊息說明需聯繫開發者

路徑逃逸由程式處理，不靠白名單

... 逃逸、符號連結指向白名單外、絕對路徑與相對路徑混用——這些一律由 backend 在解析路徑時處理（正規化後比對），**不是使用者要操心的事，介面上也不出現**。白名單管的是「哪些目錄可以納管」這個政策問題，逃逸防護是實作問題，兩者不混為一談。

7.10 UX 整理 (v0.9.0)

前述版面於各功能里程碑逐步建置，v0.9.0 專門處理跨視圖的一致性：

- **錯誤訊息全面審視**——每則訊息都包含「發生什麼」「在哪裡」「該怎麼改」。
- **載入與等待狀態**——寫出、差異掃描等操作需明確的進行中指示，避免重複點擊。
- **鍵盤操作**——樹的上下移動、Enter 展開、Esc 收合、Ctrl+S 儲存目前區塊。
- **破壞性操作的二次確認**——統一確認對話框的文案與行為。
- **空狀態**——尚無納管項目、搜尋無結果、歷史僅一筆等情境。
- **操作後的回饋與位置保持**——儲存後保留樹的展開狀態與捲動位置。

8. 版本里程碑與驗收檢查點

版本策略

每個版本都同時具備 **backend** 與 **frontend**，介面在早期版本只求可用、不求好用。這與「先做完後端再做前端」的分法不同：後者會累積五個版本沒人能實際操作的功能，缺陷要到很後面才暴露。

v0.10.0 是功能完整的目標版本。v1.0.0 不在本規劃內——待實際使用一段時間、確認功能與穩定度足夠後，再由維護者自行標記。這避免了「為了達成 1.0 而宣稱完成」的壓力。

8.1 版本能力矩陣

版本	Backend 新增	Frontend 新增	此版本結束時可以做的事
v0.1.0	清單檔讀寫與完整性檢查 git 操作 原子寫出 + 權限	身分輸入 config 清單 (唯讀, 含狀態)	在瀏覽器看到手寫清單檔的內容與各項狀態
v0.2.0	格式解析 型別記錄 歧義值警告 白名單判定 納管	檔案瀏覽器 納管流程 白名單維護	從介面把既有 config 納入管理, 內容逐位元組不變
v0.3.0	第 1 層驗證 原樣寫入 草稿層 進版 (整批原子)	參數欄位表 (基本型別) 儲存草稿 進版	改多份 config 存為草稿, 一次進版全部送出, 註解與格式不受損
v0.4.0	歷史查詢與類型過濾 退版	歷史列表 參數層級差異 退版按鈕	查一份 config 改過幾次、誰改的; 退回任一版本
v0.5.0	四種狀態判定 偏離處置	差異檢視 兩種處置	發現有人繞過流程改了檔案, 並決定覆蓋或納入
核心流程完整。此後版本專注於介面品質與驗證深度。			
v0.6.0	參數索引 (供搜尋) 單一編輯階段鎖	左側樹 雙擊展開 多 config 同時展開 參數層級搜尋	跨檔比對參數; 搜尋到參數層級; 多人誤開時被明確擋下
v0.7.0	schema 骨架生成 第 2 層驗證	欄位表套用 schema (範圍、列舉、說明)	型別與範圍錯誤在輸入當下被擋, 不需送出才知道
v0.8.0	第 3 層跨欄位規則 override 機制 規則可配置	規則警告呈現 override 理由輸入 陣列與巢狀編輯	跨欄位不合理的組合被警告; 複雜結構也能用欄位表編輯
v0.9.0	部署模式判定 閒置逾時	開發 / 部署模式差異 UX 整理六項	現場部署可用; 未受訓的同事能自行操作
v0.10.0	部署封裝 備份與復原	空狀態 說明文件入口	功能完整, 可正式投入使用

讀法

每一版都是**可完整操作的一條路徑**，不是半成品。v0.1.0 就能開瀏覽器看到東西，v0.3.0 就能改參數並套用。缺陷因此在每一版都會被實際使用暴露，而不是累積到最後。

後五版沒有新的核心流程，全部在**補驗證深度**（第 2、3 層）與**介面品質**（樹、搜尋、模式、UX）。這是刻意的順序：先讓流程能跑，再讓它好用與安全。

8.2 驗收檢查點

v0.1.0 資料層與唯讀介面

✓ 檢查點

ADR lint 可攔截：檔名格式錯誤、重號、缺 > 服務：、缺必要段落、**Status** 值非法（皆 fail），並對跳號與缺 **Alternatives** 印出警示（不擋）。可載入手寫清單檔；五種完整性錯誤（uid 重複、目標重複、來源不存在、路徑含 ..、格式未知）皆能攔截並回報具體原因。修改清單檔後寫出，**註解與欄位順序完整保留**。寫出中斷（模擬強制終止）不產生半殘檔案。瀏覽器可開啟並看到清單與各項狀態；同一功能可由 CLI 完成。

v0.2.0 納管與白名單

✓ 檢查點

能從介面納管至少三種格式的**真實 config**；納管後來源檔案與原始檔案**逐位元組相同**；正確記錄原始 owner/group/mode；hostname 自動填入且寫進檔案，重跑不因環境改變而變動。含歧義值的檔案能列出警告清單（**指名值與行號**）且阻擋匯入。**白名單**可從介面**新增與移除**；移除時若有受影響項目會列出並要求確認；納管被拒的錯誤訊息含加入白名單的入口。以 ../、符號連結兩種方式嘗試逃逸白名單皆被攔截。

v0.3.0 參數編輯與進版

✓ 檢查點

修改兩份 config 各存為草稿後，目標檔案皆未改變；按一次進版兩份一起送出，各產生一筆變更紀錄；任一份驗證失敗時整批不進版且指出是哪一份的哪一個參數。從介面修改一個 double 參數後進版，**寫出的值帶小數點**（不是整數）；**註解與未修改欄位的格式完全保留**。bool 欄位是核取方塊，結構上無法輸入非布林值。在文字層面注入 yes、Tab 縮排、尾隨空白、重複 key 四種違規皆被攔截並**指出行號**（重複 key 須列出所有出現行號）。變更紀錄的作者正確；寫出後目標的權限符合清單檔；模擬寫出失敗時變更紀錄亦回滾。

v0.4.0 歷史與退版

✓ 檢查點

連續修改同一 config 三次後可退回第一版；退版後歷史仍可見全部四筆紀錄，**歷史未被截斷**；以 cfg 類型過濾時退版紀錄不出現；目標內容確實回到第一版。差異以**參數為單位**呈現（哪個參數從什麼變成什麼），不是文字行差異。

v0.5.0 偏離偵測與處置

✓ 檢查點

以 vim 直接修改一個已納管的目標檔案，掃描後判定為**偏離且差異內容正確**；執行覆蓋後回到一致；另一份執行納入後來源更新且產生 `adopt` 紀錄；刪除一個目標後判定為未部署且可修復。四種狀態在左側樹上以色點正確彙總。

✓✓ 核心流程完成檢查點

v0.1.0–v0.5.0 全數通過。**能以純介面完成一次完整週期**：納管 → 修改 → 退版 → 偵測偏離 → 處置；**同樣的週期也能以純 CLI 完成**。整合測試對真實 git repo 執行且全數通過；`core/` 的單元測試在無檔案系統、無 git 的環境下全數通過。

v0.6.0 樹、搜尋與編輯鎖

✓ 檢查點

左側樹依 hostname 與群組正確分層，父節點彙總最嚴重狀態；雙擊展開參數，**可同時展開三個以上 config 並各自儲存**；搜尋 `max_vel` 能列出所有含該參數的 config 並在展開時定位到該列；搜尋 `0.55` 能找到值符合的參數。**開第二個分頁時顯示唯讀並指名持有者與開始時間**；持有者關閉分頁後第二個分頁可取得編輯權；異常中斷後階段於逾時內自動釋放。

v0.7.0 Schema 驗證

✓ 檢查點

對一份已納管的 config 產生 schema 骨架；把 **double 參數改為整數在輸入當下即被擋**；有範圍限制的參數超出範圍時即時標示；schema 的說明文字顯示在欄位旁。**同一份 schema 能驗證 YAML、TOML、JSON 三種格式的 config**（驗證的是解析後的資料結構）。

v0.8.0 跨欄位規則與複雜結構

✓ 檢查點

違反 `min < max` 類規則時顯示警告且 override 需填理由，理由出現在變更紀錄；規則檔的修改能實際改變驗證行為，且退版時規則一併回退。**陣列可從介面增刪元素並調整順序**；巢狀物件以可折疊區塊呈現且縮排由系統產生。型別未知的項目退回文字區塊並**明確標示為未結構化編輯**。

v0.9.0 部署模式與 UX

✓ 檢查點

部署模式下**不顯示「記住此裝置」**，閒置 10 分鐘自動退出並釋放編輯階段，剩餘 1 分鐘時提示可延長；模式由 backend 判定，前端改動無法繞過。**由未參與開發的同事在無人指導下完成**：納管一份新 config、修改並儲存、故意輸入錯誤值並依錯誤訊息自行修正、查看歷史並退版、處理一次偏離。全程無需詢問操作方式。

v0.10.0 功能完整

✓✓ 發布檢查點

以**實際專案的至少 10 份 config** 完成納管，連續使用兩週無資料遺失；期間所有修改皆可從變更紀錄追溯至具體人員與時間；至少完成一次真實的退版；至少偵測到一次真實的偏離並正確處置；容器重啟後狀態完整保留；備份與復原程序實際演練過一次。

v1.0.0 由維護者自行標記

v0.10.0 通過後即可投入正式使用。**v1.0.0 不設在本規劃內**——待實際運行一段時間、確認功能覆蓋與穩定度足夠後再標記，避免為了達成版號而宣稱完成。

附錄 A. 導入組織共用容器模板

本附錄不在 v0.10.0 範圍

本專案初期不引入模板。本附錄記錄對齊策略與日後導入時的遷移工作，確保現在的設計不會擋住它。

A.1 對齊策略：三類決策

對齊基準：模板 v0.42.0

依據是模板初始化腳本內的安裝路徑清單（權威定義），而非組織內既有 downstream repo 的目錄形狀——抽查兩個 downstream，分別停在 v0.12.1 與 v0.41.0，其結構已被取代（詳見 § 3.3.5）。

本專案所屬的組織有一套共用的容器模板（下稱「模板」），以 subtree 方式導入進各專案，提供容器 lifecycle (init/restart/watchdog/log persistence)、設定解析與渲染、CI worker、測試 dispatcher 等共用機制。

本專案初期不引入該模板——維護人員不同，且專案早期不需要完整 lifecycle。代價是模板提供的東西要自己做一份。因此關鍵在於分辨哪些現在就該對齊、哪些該做最小版本、哪些現在做反而是浪費。判準是兩個成本的比較：現在做的成本，與日後導入時改的成本。

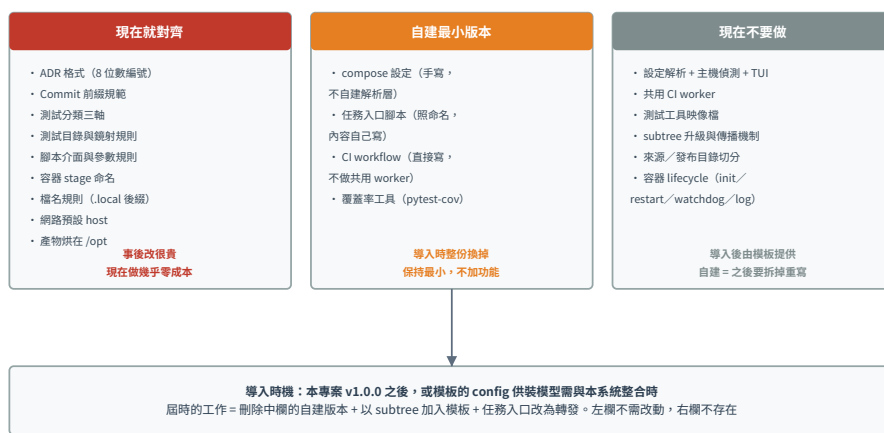


圖 16 | 三類決策：現在對齊／自建最小版／現在不做

三類決策的判準

類別	判準	處理方式
現在就對齊	現在做幾乎零成本，但事後改需要重寫歷史、重命名檔案、或重構目錄結構	直接採用模板的規則。導入時不需任何改動
自建最小版本	現在必須有，否則專案跑不起來；但模板已有更完整的版本	做到「剛好夠用」即停，不加功能。導入時整份刪除，換成模板的
現在不要做	現在做成本高，且導入後模板會直接提供	不做。若真的需要，先評估是否應提前導入模板

最容易犯的錯

把中欄的東西做大。自建的 `compose.yaml` 一旦長出主機偵測、多語系預設、TUI 編輯，它就變成一個競爭版本的設定解析層——導入時要嘛拆掉重做，要嘛兩套並存（直接違反不變式 6）。中欄的東西只能修 bug，不能長功能；需要新功能時，那是提前導入模板的訊號。

A.1.1 現在即對齊的架構細節

以下項目屬「現在就對齊」類，導入時不需改動。

Dockerfile stage 命名

採用模板保留的 baseline stage 名稱，使日後導入時 Dockerfile 不需重構：

Stage	性質	說明
sys	建置中介	系統層依賴，無可執行服務
devel-base	建置中介	開發工具鏈
devel	session	開發用互動 shell；bind source；永不作為部署目標，不發 restart policy
runtime	可部署	現場成品；config 烘入 + 掛載覆蓋；發 restart policy
runtime-test	session	建置時的 smoke 測試；跑完即結束，不發 restart policy

可部署的判定必須是單一述詞（一個函式），不得在多處各自判斷——兩處判斷遲早會不一致，而不一致的方向是「不該部署的被部署了」。

產物烘焙位置

自建的產物一律烘在絕對路徑 `/opt`，不在 `$HOME`。

理由：`ENV HOME` 在建置時就解析完成，任何烘在 `$HOME` 底下的東西都與建置時的使用者名稱綁死，在重建、pull 映像、或 `docker save / load` 到不同使用者的機器上就會壞掉——而且是幾個月後在唯一重要的那台機器上壞。`~/x → /opt/x` 可以留一個純為好找的 symlink，但不得有任何程式 source 它。

可檢查的規則：路徑中不得出現具體使用者名稱。

檔名規則

標準名字是工具的、會被重新產生；`.local` 後綴標記操作者的變體、永不被改寫。此規則對 `Dockerfile`、`compose.yaml`、`.env` 家族一體適用，本專案自建的設定檔亦然。

容器網路

預設 `host`，`bridge` 為 opt-in（不變式 4）。`bridge` 作為預設會靜默破壞跨機器的 ROS 通訊——這正是「方便的預設可能無聲破壞真實部署」的典型案例。

A.1.2 導入時的遷移工作

1. 以 `subtree` 加入模板，執行其初始化腳本。
2. 刪除自建的 `compose.yaml`，改以模板的設定檔描述，由模板渲染。
3. `justfile` 改為 thin forwarder，內容指向模板的共用目錄。
4. CI workflow 改呼叫模板的 reusable worker。
5. 測試目錄與分類**不需改動**（已對齊），僅需接上模板的 dispatcher。
6. ADR 目錄與編號**不需改動**（已對齊），僅需接上模板的 numbering lint。

A.2 開發與現場的供裝分離

v2.0.0／v3.0.0

此項在導入組織共用容器模板之前不適用，開發初期不處理。列於此處是為了確保現在的設計不會擋住它。

模板定義了一套 config 供裝模型：開發時 config 以 bind mount 進容器、直接編輯即生效；可部署階段則把 config 烘進映像作為預設，再加一個「掛檔案覆蓋它」的掛勾，使部署不需重建映像即可調整。開發者與操作者的分界依 git 追蹤狀態決定：已提交的是開發者預設，未追蹤的是操作者覆蓋層（掛載，預設唯讀）。命名上，標準名字屬於工具且會被重新產生，`.local` 後綴屬於操作者且永不被改寫。

A.2.1 導入時必須處理的邊界

模板的設定產生器會擁有一組檔案（`.setup.conf`、`.env`、生成的 `compose.yaml` 等）。屆時 **config-manager 必須把這些檔名列入納管黑名單**——若兩套機制都宣稱對同一份檔案有 ownership，就有兩個真實來源，違反不變式 6。

兩者管理的是不同層次：模板管容器自身的供裝路徑，config-manager 管主機上散落的應用層 config。導入時應以 ADR 記錄此邊界。

附錄 B. v0.10.0 之後的功能規劃

本章項目均不在 v0.10.0 範圍

列出以確保架構設計不排除這些方向，但不影響 v0.10.0 的交付。

B.1 Schema 自動生成 (ROS 2)

ROS 2 節點的參數型別資訊已存在於執行中的 node：

```
ros2 param list
|
▼
ros2 param describe /<node> <param>
|   ParameterDescriptor:
|     type / description / floating_point_range / integer_range / read_only
|
▼
轉換為 JSON Schema → 存入 .schemas/ → 供 lint 與 Form view 使用
```

優點：schema 自動與程式碼同步，不會脫節；range constraint 可直接轉為 UI 的 min/max 限制。

前提：node 的 `declare_parameter` 需帶完整 `ParameterDescriptor`。若僅宣告預設值則只能取得型別，無法取得範圍。第三方 package (如 Nav2) 通常有完整宣告，自研 node 需確認。

```
// 僅有型別資訊
declare_parameter("max_vel_x", 0.5);

// 完整資訊
rcl_interfaces::msg::ParameterDescriptor desc;
desc.description = "最大前進速度";
desc.floating_point_range.push_back(range); // min / max / step
declare_parameter("max_vel_x", 0.5, desc);
```

B.2 ROS 2 Install Space 偵測

問題：ROS 2 的 config 常置於 package 內。colcon build 會將 `src/` 的檔案複製至 `install/<pkg>/share/<pkg>/`，而 launch file 透過 `get_package_share_directory()` 讀取的是 install 空間那份。若納管 install 空間的檔案，下次 build 會被覆蓋，且屬靜默失效。

術語澄清

ROS 2 的 colcon 只有 `build/`、`install/`、`log/`，沒有 **devel space** (devel 是 ROS 1 catkin 的概念)。實務上有意義的區分是：**開發機同時有 `src/` 與 `install/`，部署機通常只有 `install/`**。另需注意 `colcon build --symlink-install` 會使 install 空間的檔案本身即為指向 `src/` 的 symlink，此時寫入 install 等同寫入 `src/`。

v0.10.0 的處理：不做特別偵測，由使用者自行指定路徑並自行負責。

未來版本的判斷邏輯：

情境	判定	處置
路徑不在 install space	一般檔案	正常納管
在 install space 且存在對應 <code>src/</code>	開發環境	警告：build 會覆蓋。建議改納管 <code>src/</code> 來源檔，或將 config 移出 package
在 install space 且無對應 <code>src/</code>	部署環境	正常納管。無 source code 即不會被 rebuild 覆蓋，是安全的納管對象
檔案本身為 symlink 指回 <code>src/</code>	<code>--symlink-install</code>	提示：實際寫入的是 <code>src/</code> 。建議直接納管 symlink 目標路徑

檢查項	方法
是否為 install space	路徑含 <code>install/<pkg>/share/<pkg>/</code> 結構，或上溯尋找 <code>install/setup.bash</code> 、 <code>install/.colcon_install_layout</code>
是否存在 source	上溯至 workspace 根目錄，檢查 <code>src/<pkg>/</code> 是否存在
是否 symlink-install	對該檔案執行 <code>test -L</code>

長期建議：將 config 移出 ROS package，置於獨立位置，launch file 改吃絕對路徑或環境變數。可根本消除上述所有情境判斷。

B.3 其他 Backlog

項目	說明
定期背景掃描與通知	以 cron/systemd timer 定期執行差異偵測，發現 偏離時主動通知（可接 Telegram）。v0.10.0 僅提供手動觸發
提權寫入	/etc 等路徑需 root 權限。以 sudoers 白名單限定特定 apply 操作，服務本身不以 root 執行
完整匯入檢視	納管前的詳細檢查介面：逐欄位型別對照表、解析後的結構樹、與原檔並排比對、批次匯入預覽。v0.10.0 前只做歧義值警告（見 § 5.1）
歷史顯示的細部切換	讓使用者自選要看哪幾類變更。v0.10.0 前只做「只看內容變更／全部」兩檔，因為更細的分類在實際使用前無從得知哪些組合有用
差異的完整檔案檢視	除「只看差異」外，加入列出全部參數、差異處標色的模式。價值要到 config 變大之後才顯現
搜尋歷史	保留最近 10 筆含選定範圍，存於瀏覽器端。需先確認搜尋語法穩定
角色的真正權限控管	目前為自我宣告。若需要驗證，應在反向代理層處理並由 backend 回報角色，不在本系統實作認證
內容全文搜尋	跨所有納管 config 搜尋值或 key，回傳 (file, json_path, value) 三元組
多機器分發	Pull 模型（各機器自行同步）優於 push。可評估以 Ansible 作為分發層
三層 overlay	base/model/device 三層 config 疊加，處理跨案場差異。參考 Karelics param_configuration 的資料模型
跨機 config diff matrix	並排比較多台機器的同名參數，高亮偏離 baseline 者。自建系統相對於現成工具的主要差異化價值
多人協作	編輯鎖定或樂觀鎖。目前假設單一使用者同時操作

概念：跨機 diff matrix

	amr-01	amr-07	forklift-03
max_vel_x	0.8	0.8	0.5 ▲
inflation_r	0.55	0.35 ▲	0.55
scan_topic	/scan	/scan	/scan

多車運行時「為何這台與那台行為不同」是最常見且最難查的問題，git diff 只能兩兩比較，缺乏結構化對照。

附錄 C. 參考專案評估

C.1 Config／Dotfiles 管理工具

專案	優點	缺點	是否採用與原因
chezmoi Go	Copy + apply 模式成熟；template 處理跨機器差異；支援加密與 secret；跨平台；文件完整	CLI-only，無 API 供後端整合；檔案命名慣例（dot_ 前綴）；預設目標為 \$HOME	借鏡模式，不直接採用。 本系統採用其 copy + apply 核心設計。不直接使用的原因：以 \$HOME 為中心，管理 /opt、/etc 需 workaround；缺乏程式化介面，包裝 CLI 的錯誤處理會很脆弱
GNU Stow Perl	極簡；概念只有 symlink；完全可逆	Symlink 三大失效模式（§ 2.1）；不處理權限；無 template	否決。 Symlink 機制與 Docker 部署根本衝突
yadm Shell	Bare repo，檔案完全不搬動；零侵入；支援加密與 alternate files	work-tree=/ 使 reset 與 branch 切換語意危險；批次操作受限	否決。 不利整批進退版（§ 2.1 方案 B）
Ansible Python	寫等；template 完整；inventory 分群；-check dry run；多機分發成熟	對單機 config 管理過重；playbook 本身也需版控維護；無 config 內容層級的驗證	未來多機分發時再評估。 v1 階段引入會增加不必要的複雜度
Karelics / param_configuration	ROS 2 參數的三層 overlay；!overlay tag 與 placeholder 求值機制成熟	僅處理 ROS 2 參數；無 UI；無版控整合	三層 overlay 的資料模型參考 （附錄 B.3）。設計時應先研讀其實作

C.2 參數編輯的元件選項

前提：不做文字編輯器

§ 7.5 已定案介面以**型別欄位表**編輯參數，不提供檔案文字編輯（ADR-0000013）。因此本節評估的是**表單／參數面板**元件，不是程式碼編輯器。

專案	優點	缺點	是否採用
@json-editor/ json-editor Vanilla JS	由 schema 直接生成表單——本專案已有 schema； 處理陣列與巢狀物件 ；每欄位的驗證錯誤呈現；純 JS 可直接引入， 無建置流程	外觀較舊，需自訂樣式；僅支援 JSON Schema 較早的草案版本；維護節奏較慢	採用，作為起步。 與需求對得最準——schema 驅動、支援陣列與巢狀、有欄位級驗證呈現、符合單一 HTML 的決定
Tweakpane lil-gui（同類）	參數面板的視覺語言最接近；巢狀資料夾；無相依可直接引入；成熟穩定	沒有陣列綁定 （支援型別僅數值、字串、布林、色彩、二至四維點；同類函式庫作者亦指出這幾套都無法編輯陣列）； 沒有每欄位的驗證錯誤呈現	不採用。 缺陣列與驗證呈現兩項，而它換來的拉霸與即時監看正是本專案不需要的（§ 7.5.2 明訂不用拉霸）。補陣列要寫外掛，那不如自建
自建渲染器	完全符合。可精確控制「目前值／來源值／驗證狀態」三欄的呈現	需自行實作與維護，估計數百行；陣列與巢狀的邊界案例要自己處理	備案。 若「目前值／來源值」並列或差異標示無法在 json-editor 內做出來則改為自建，屆時可借用 Tweakpane 的資料夾與繫結模型作為結構參考
JSONForms	Schema 與 UI Schema 分離，layout 可精細控制；企業級維護	綁定前端框架，與單一 HTML 的決定衝突；需額外撰寫 UI Schema	不採用。 框架相依抵觸 § 3.2 的前端形態決定
MetaConfigurator	現成的 schema editor + form generator；具 schema 推斷功能，與附錄 B.1 需求吻合	是完整應用而非函式庫，嵌入需 fork 或 iframe；客製化受限	不採用為元件，但值得參考。 其 schema 推斷邏輯可作為 T12 的實作參考
monaco-yaml CodeMirror 6	程式碼編輯器，schema 驅動的即時驗證	是文字編輯器，與 ADR-0000013 的方向相反	不採用。 介面不提供檔案文字編輯。 僅在型別未知、結構無法表格化的項目退回文字區塊時 可能需要（§ 7.5.4），屆時基本 textarea 已足夠

C.3 解析與驗證函式庫

函式庫	語言	用途與優點	限制
tomlkit	Python	TOML round-trip，保留註解與格式。 config 清單檔讀寫使用	僅 TOML
ruamel.yaml	Python	YAML round-trip，保留註解、引號樣式、鍵順序；可指定 1.1／1.2	API 較 PyYAML 複雜；效能略低（本場景無影響）
configobj	Python	INI 解析且 保留註解	標準庫 configparser 不保留註解故需替代；INI 無標準規格，方言差異大
eemeli/yaml	JavaScript	前端 YAML round-trip；yaml-language-server 的底層 parser	—
pydantic	Python	config 清單檔的結構驗證與型別強制；與 FastAPI 整合	不處理 round-trip，需與 tomlkit 併用
jsonschema	Python	後端 schema 驗證，支援 draft 2020-12	錯誤訊息對非技術使用者不夠友善，需二次包裝
ajv	JavaScript	前端即時驗證，效能佳	—
yamlLint	CLI／Python	格式一致性檢查（縮排、行長、尾隨空白、重複 key），與 § 6.3 部分規則重疊	不做型別或 schema 驗證，僅覆蓋第 1 層的一部分

C.4 已評估但不採用的替代路線

方案	若採用可得到什麼	不採用的原因
code-server (VSCode in browser)	編輯器、workspace 分群、split editor、跨檔搜尋、yaml-language-server schema 驗證、Git 整合，全部現成且免維護	覆蓋約七成需求，但缺少關鍵三項：(1) 無法作為 gate——LSP 錯誤不阻擋存檔；(2) 無 config 清單檔概念，不知道哪些檔案該被管理，也無 寫出機制；(3) 無法呈現 target 與 repo 的差異狀態。 建議：若本系統開發受阻，code-server + 一份寫好的 JSON Schema 是合理的過渡方案
純 Git + CLI 腳本	成本最低，無 UI 維護負擔	無法達成「統一輸入介面」與「非工程角色可操作」的目標。但 backend 本身設計為可獨立以 CLI 呼叫（§ 3.3），保留此路徑的價值
Kubernetes ConfigMap 類機制	成熟的 config 版本化與滾動更新	需引入 K8s 或等價編排層，對單機／少量機器的機器人部署過重
GitPython／pygit2	程式化的 git 操作，型別安全	本系統 git 操作種類少且固定。CLI 包裝的行為與人工操作完全一致，除錯時可直接複製指令重現；函式庫抽象在 revert、log 過濾等場景反而增加不確定性

本文件為技術討論用途，記錄設計決策與其理由。已否決的方案一併保留，供後續討論時參考避免重複評估。圖表原始碼另附 diagrams.md。